

MoE Models from Scratch

Phase 5 section-development skeleton

MiniDeepSeekMoE section brief scaffold with standalone artifact placeholders

Every figure, table, listing, and equation below is included via `\input`.

Contents

1	From dense compute to conditional compute	1
1.1	The dense FFN bottleneck	1
1.2	Conditional compute as the central idea	2
1.3	A tiny routing story	3
1.4	What from scratch means for this book	4
1.5	The MiniDeepSeekMoE build ladder	5
1.6	Chapter summary and handoff	6
2	A dense Transformer baseline from scratch	9
2.1	Dataset and token batches	9
2.2	Token and position embeddings	10
2.3	Causal self-attention as the context path	11
2.4	Dense feed-forward block as the replacement target	12
2.5	Training loop and sampling baseline	13
2.6	Baseline diagnostics and comparison slots	14
3	Experts, routers, and the first MoE layer	17
3.1	Expert MLPs as named FFNs	17
3.2	Router scores and tensor shapes	18
3.3	Top-k selection without dispatch	19
3.4	Naive token-by-token dispatch	20
3.5	Shape and gradient checks	21
3.6	Chapter summary and handoff	22
4	Top-1 routing and Switch-style dispatch	25
4.1	Why top-1 routing is the simplest sparse case	25
4.2	Assignment masks and expert buckets	26
4.3	Dispatching tokens to selected experts	27
4.4	Capacity and dropped tokens	28
4.5	Load histograms and failure modes	29
4.6	Chapter summary and handoff	30
5	Top-2 routing and weighted expert combination	33
5.1	Why top-2 changes the routing story	33
5.2	Normalizing selected router scores	34
5.3	Dispatch for two expert paths per token	35
5.4	Comparing top-1 and top-2 diagnostics	36
5.5	Numerical stability in top-k gates	37
5.6	Chapter summary and handoff	39
6	Vectorized dispatch, capacity, and routing efficiency	41
6.1	Flattened tokens as the dispatch unit	41
6.2	Expert batch construction	42
6.3	Capacity factors and overflow policy	43
6.4	Unpacking and weighted combine	44

6.5	Equivalence tests against the naive layer	45
6.6	Chapter summary and handoff	46
7	Router training and load-balancing losses	49
7.1	Expert imbalance as a training problem	49
7.2	Auxiliary load-balancing loss	50
7.3	Integrating auxiliary loss into training	52
7.4	Router score scale and z-loss	53
7.5	Routing dashboards and training logs	54
7.6	Chapter summary and handoff	55
8	Shared experts and fine-grained routed experts	57
8.1	The role of shared experts	57
8.2	Fine-grained routed expert segmentation	58
8.3	The MiniDeepSeekMoE layer contract	59
8.4	Combining shared and routed outputs	60
8.5	Specialization diagnostics	61
8.6	Chapter summary and handoff	62
9	Auxiliary-loss-free load balancing	65
9.1	The limitation of auxiliary balancing	65
9.2	Router bias as non-trainable state	66
9.3	Biased selection and unbiased combine	67
9.4	Dynamic bias update from expert load	68
9.5	Comparing balancing methods	69
9.6	Chapter summary and handoff	70
10	Assembling MiniDeepSeekMoE end to end	73
10.1	Configuration objects and model variants	73
10.2	Decoder block with dense prefix and MoE layers	74
10.3	Forward pass outputs and loss dictionary	75
10.4	Training and sampling scripts	76
10.5	Full-model smoke tests	77
10.6	Chapter summary and handoff	78
11	Training, diagnostics, and controlled experiments	81
11.1	Experiment design for a from-scratch book	81
11.2	Dataset preparation and reproducibility	82
11.3	Loss curves and routing dashboards	84
11.4	Ablation matrix: dense, top-1, top-2, shared, and bias-balanced	85
11.5	Interpreting expert behavior carefully	87
11.6	Chapter summary and handoff	88
12	Inference, performance, and scaling beyond the toy model	91
12.1	Autoregressive inference with sparse FFNs	91
12.2	Batching routed tokens at inference	92
12.3	Compute, memory, and active-parameter accounting	93
12.4	Expert parallelism and all-to-all communication	94
12.5	Mapping the book model to production MoE families	96
12.6	Book summary and next steps	97
A	LaTeX artifact and diagram rules	99
B	Tensor shape checklist	101
C	Codebase smoke tests and reproducibility checklist	103

Chapter 1

From dense compute to conditional compute

This chapter covers

- Why dense Transformer feed-forward layers dominate parameter and compute growth.
- How conditional compute changes the cost story without changing the decoder interface.
- The MiniDeepSeekMoE roadmap that the rest of the book implements.

Motivate MoE from the dense FFN bottleneck and establish the build ladder.

1.1 The dense FFN bottleneck

Show that the dense feed-forward block is the natural place to introduce sparsity because every token pays for every hidden unit.

Use a four-token batch with `d_model=8` and an FFN expansion factor of 4; count multiply-adds for all tokens. The planned mechanics should introduce the equation before the implementation listing.

$$\text{Dense FFN cost estimate using } D, H, \text{ and number of tokens } N. \quad (1.1)$$

Listing 1.1: Planned code placeholder for the dense FFN bottleneck.

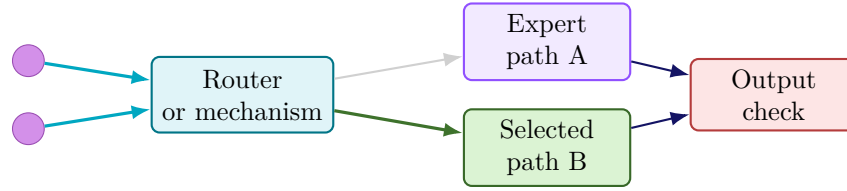
```
def the_dense_ffn_bottleneck_placeholder(x, config):
    """Chapter 1.1 placeholder.

    Planned purpose: Tiny function that estimates dense FFN parameters and per-token matrix multiplies.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

A small cost table proving that increasing H increases every token path.

Once the bottleneck is visible, the next question is whether every token really needs the same FFN.



Ch. 1.1: The dense FFN bottleneck

Figure 1.1: Dense decoder block with the FFN path highlighted as the always-on compute region.

Table 1.1: Parameter and activation shape comparison for attention, dense FFN, and the residual path.

Item	Planned content	Status
Mechanism	Show that the dense feed-forward block is the natural place to introduce sparsity because every token pays for every hidden unit.	Placeholder
Mini-example	Use a four-token batch with $d_{\text{model}}=8$ and an FFN expansion factor of 4; count multiply-adds for all tokens.	Placeholder
Diagnostic	A small cost table proving that increasing H increases every token path.	Placeholder

1.2 Conditional compute as the central idea

Introduce MoE as selective FFN computation rather than a mysterious new model family.

Route four tokens to two of four available experts and compare activated parameters against total parameters.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Activated-parameter view: total expert bank versus top-k expert use per token.} \quad (1.2)$$

Listing 1.2: Planned code placeholder for conditional compute as the central idea.

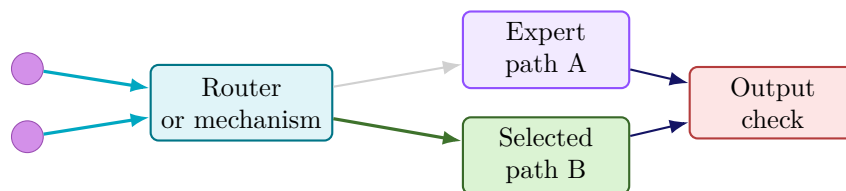
```
def conditional_compute_as_the_central_idea_placeholder(x, config):
    """Chapter 1.2 placeholder.

    Planned purpose: Pseudocode that chooses expert IDs for tokens and reports active expert count.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Compute the active fraction for $E=4, K=2$ and $E=16, K=2$.

Selective compute creates a new component: the router, which needs its own tensor contract.



Ch. 1.2: Conditional compute as the central idea

Figure 1.2: Tokens split through a router into a sparse subset of experts, then recombine.

Table 1.2: Dense versus conditional-compute vocabulary: FFN, expert, router, active parameters, total parameters.

Item	Planned content	Status
Mechanism	Introduce MoE as selective FFN computation rather than a mysterious new model family.	Placeholder
Mini-example	Route four tokens to two of four available experts and compare activated parameters against total parameters.	Placeholder
Diagnostic	Compute the active fraction for E=4, K=2 and E=16, K=2.	Placeholder

1.3 A tiny routing story

Give readers a concrete token-by-token routing example before any model code appears.

Use tokens [cat, sat, code, runs], four experts, and top-2 routing weights.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Weighted expert output } y_t = \sum \text{gate}_i \text{expert}_i(x_t). \quad (1.3)$$

Listing 1.3: Planned code placeholder for a tiny routing story.

```

def a_tiny_routing_story_placeholder(x, config):
    """Chapter 1.3 placeholder.

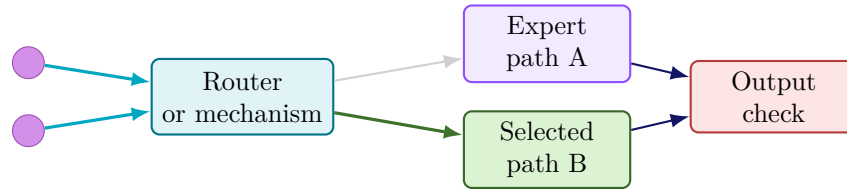
    Planned purpose: Manual PyTorch tensor example that applies torch.topk and normalized weights.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Check that gates for each token sum to one after top-k normalization.

This toy flow becomes the shape contract for every later implementation.



Ch. 1.3: A tiny routing story

Figure 1.3: A storyboard of router scores, selected experts, expert outputs, and weighted combine.

Table 1.3: Toy router-score matrix with top-2 expert choices and normalized gates.

Item	Planned content	Status
Mechanism	Give readers a concrete token-by-token routing example before any model code appears.	Placeholder
Mini-example	Use tokens [cat, sat, code, runs], four experts, and top-2 routing weights.	Placeholder
Diagnostic	Check that gates for each token sum to one after top-k normalization.	Placeholder

1.4 What from scratch means for this book

Define the implementation boundary: small readable modules, explicit tensors, and diagnostics before optimization.

Contrast a one-line library MoE call with a transparent router, dispatch, expert, and combine pipeline.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Interface invariant: MoE layer maps } (B,T,D) \text{ to } (B,T,D). \quad (1.4)$$

Listing 1.4: Planned code placeholder for what from scratch means for this book.

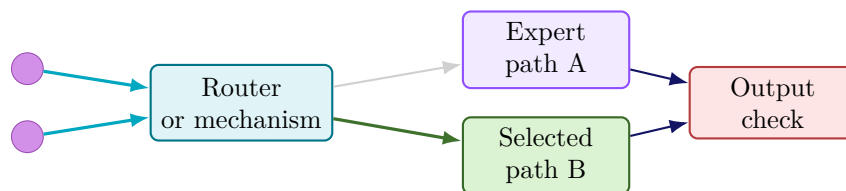
```
def what_from_scratch_means_for_this_book_placeholder(x, config):
    """Chapter 1.4 placeholder.

    Planned purpose: Skeleton class layout for Router, ExpertMLP, MoELayer, and MiniDeepSeekMoE.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Checklist that every chapter must include shapes, a code path, and a verification artifact.

With the boundary defined, we can plan the exact model we will build.



Ch. 1.4: What from scratch means for this book

Figure 1.4: Layered build approach from scalar example to vectorized PyTorch to training script.

Table 1.4: Included and excluded scope for the book, including distributed training and production kernels.

Item	Planned content	Status
Mechanism	Define the implementation boundary: small readable modules, explicit tensors, and diagnostics before optimization.	Placeholder
Mini-example	Contrast a one-line library MoE call with a transparent router, dispatch, expert, and combine pipeline.	Placeholder
Diagnostic	Checklist that every chapter must include shapes, a code path, and a verification artifact.	Placeholder

1.5 The MiniDeepSeekMoE build ladder

Preview the book path from dense baseline to shared experts, fine-grained experts, and router-bias balancing.

Use the smoke-test configuration with two layers, four routed experts, and one shared expert.

The planned mechanics should introduce the equation before the implementation listing.

Final layer formula combining residual, shared expert output, and routed expert output. (1.5)

Listing 1.5: Planned code placeholder for the MiniDeepSeekMoE build ladder.

```

def the_minideepseekmoe_build_ladder_placeholder(x, config):
    """Chapter 1.5 placeholder.

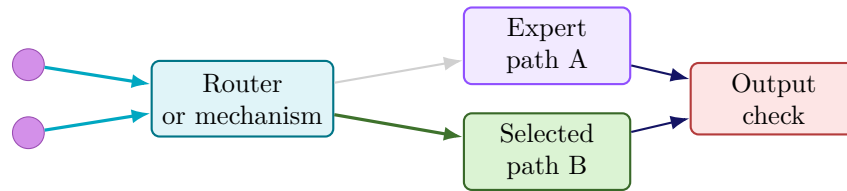
    Planned purpose: Configuration dictionary for MiniDeepSeekMoE-Smoke and MiniDeepSeekMoE-16x2.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Milestone checklist that marks which diagnostics appear by each chapter.

The first build step is a dense Transformer baseline that gives the MoE layer somewhere to live.



Ch. 1.5: The MiniDeepSeekMoE build ladder

Figure 1.5: Roadmap ladder: dense baseline, top-1, top-2, capacity, auxiliary loss, shared experts, bias balancing, full model.

Table 1.5: Chapter-by-chapter implementation milestones and expected code outputs.

Item	Planned content	Status
Mechanism	Preview the book path from dense baseline to shared experts, fine-grained experts, and router-bias balancing.	Placeholder
Mini-example	Use the smoke-test configuration with two layers, four routed experts, and one shared expert.	Placeholder
Diagnostic	Milestone checklist that marks which diagnostics appear by each chapter.	Placeholder

1.6 Chapter summary and handoff

Close the motivation chapter with the vocabulary and constraints the reader will reuse.

Revisit the four-token example and label every object with its future tensor name.

The planned mechanics should introduce the equation before the implementation listing.

Shape summary for x , router-logits, topk-idx, gates, and y . (1.6)

Listing 1.6: Planned code placeholder for chapter summary and handoff.

```

def chapter_summary_and_handoff_placeholder(x, config):
    """Chapter 1.6 placeholder.

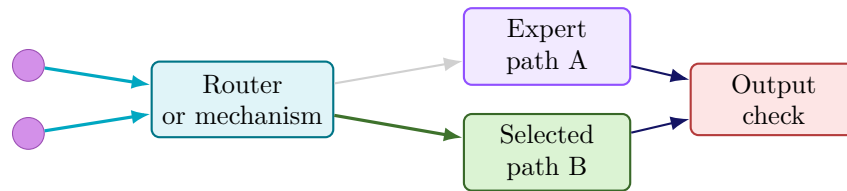
    Planned purpose: Sanity-check function signatures that later chapters must satisfy.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Reader checkpoint: identify which part of the dense model will be replaced by MoE.

Next we build the dense decoder baseline so the replacement is meaningful.



Ch. 1.6: Chapter summary and handoff

Figure 1.6: Concept map linking tokens, router scores, top-k experts, gates, and combined output.

Table 1.6: Glossary of first-use terms for the rest of the book.

Item	Planned content	Status
Mechanism	Close the motivation chapter with the vocabulary and constraints the reader will reuse.	Placeholder
Mini-example	Revisit the four-token example and label every object with its future tensor name.	Placeholder
Diagnostic	Reader checkpoint: identify which part of the dense model will be replaced by MoE.	Placeholder

Chapter 2

A dense Transformer baseline from scratch

This chapter covers

- The tensor path through embeddings, causal attention, dense FFN, residuals, and logits.
- A compact PyTorch baseline that trains and samples on a small dataset.
- The baseline diagnostics needed to compare dense and sparse variants later.

Build a minimal decoder-only language model before replacing the FFN with MoE.

2.1 Dataset and token batches

Create the small next-token prediction pipeline used for every model variant.

Load a short text corpus, tokenize it, and create B=2, T=8 training examples.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Next-token objective index relation } \text{target}[t] = \text{input}[t+1]. \quad (2.1)$$

Listing 2.1: Planned code placeholder for dataset and token batches.

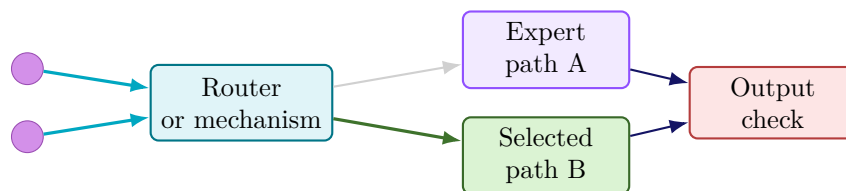
```
def dataset_and_token_batches_placeholder(x, config):
    """Chapter 2.1 placeholder.

    Planned purpose: Minimal get_batch function with input and target shifts.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Print the first input-target pair and decode it to verify shifting.

Token IDs become vectors through embeddings and positions.



Ch. 2.1: Dataset and token batches

Figure 2.1: Text to token IDs to input-target shifted batches.

Table 2.1: Batch tensor shapes and dataloader responsibilities.

Item	Planned content	Status
Mechanism	Create the small next-token prediction pipeline used for every model variant.	Placeholder
Mini-example	Load a short text corpus, tokenize it, and create B=2, T=8 training examples.	Placeholder
Diagnostic	Print the first input-target pair and decode it to verify shifting.	Placeholder

2.2 Token and position embeddings

Implement the entry point that maps integer token IDs to dense vectors.

Use `vocab_size=32`, `block_size=8`, and `d_model=16` to inspect embedding shapes.

The planned mechanics should introduce the equation before the implementation listing.

$$x = \text{token-embedding}(\text{idx}) + \text{position-embedding}(\text{pos}). \quad (2.2)$$

Listing 2.2: Planned code placeholder for token and position embeddings.

```

def token_and_position_embeddings_placeholder(x, config):
    """Chapter 2.2 placeholder.

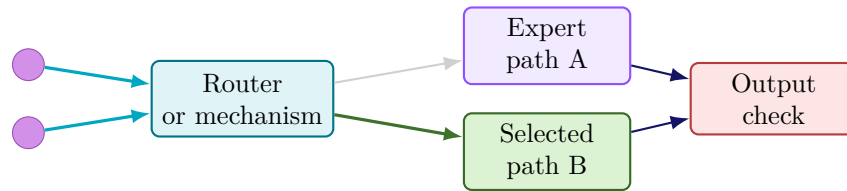
    Planned purpose: Embedding module forward pass returning x with shape (B,T,D).
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Assert output shape and confirm gradients flow to both embedding tables.

The embedded sequence now enters causal self-attention.



Ch. 2.2: Token and position embeddings

Figure 2.2: Token IDs and position IDs summed into the model input tensor.

Table 2.2: Embedding parameter counts and output shapes.

Item	Planned content	Status
Mechanism	Implement the entry point that maps integer token IDs to dense vectors.	Placeholder
Mini-example	Use vocab-size=32, block-size=8, and d-model=16 to inspect embedding shapes.	Placeholder
Diagnostic	Assert output shape and confirm gradients flow to both embedding tables.	Placeholder

2.3 Causal self-attention as the context path

Build enough attention to make the baseline a real decoder while keeping MoE focus on the FFN.

Trace a single attention head with B=1, T=4, D=8.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Masked attention } \text{softmax}(\mathbf{QK}^T / \sqrt{d\text{-head}})\mathbf{V}. \quad (2.3)$$

Listing 2.3: Planned code placeholder for causal self attention as the context path.

```

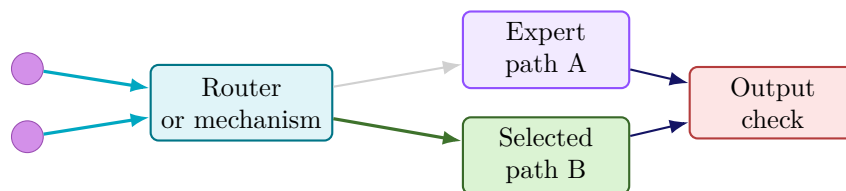
def causal_self_attention_as_the_context_path_placeholder(x, config):
    """Chapter 2.3 placeholder.

    Planned purpose: Compact multi-head causal attention module.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
  
```

Checkpoint

Check that a token cannot attend to future positions.

After attention mixes context, the dense FFN transforms each token independently.



Ch. 2.3: Causal self-attention as the context path

Figure 2.3: Causal mask applied to attention scores before softmax.

Table 2.3: Q, K, V, score, probability, and output shapes.

Item	Planned content	Status
Mechanism	Build enough attention to make the baseline a real de-coder while keeping MoE focus on the FFN.	Placeholder
Mini-example	Trace a single attention head with B=1, T=4, D=8.	Placeholder
Diagnostic	Check that a token cannot attend to future positions.	Placeholder

2.4 Dense feed-forward block as the replacement target

Implement the dense FFN in a way that makes the later expert replacement obvious.

Apply the same MLP to every token in a B*T flattened view.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{FFN}(x) = W_2 \text{ activation}(W_1 x + b_1) + b_2. \quad (2.4)$$

Listing 2.4: Planned code placeholder for dense feed forward block as the replacement target.

```

def dense_feed_forward_block_as_the_replacement_target_placeholder(x, config):
    """Chapter 2.4 placeholder.

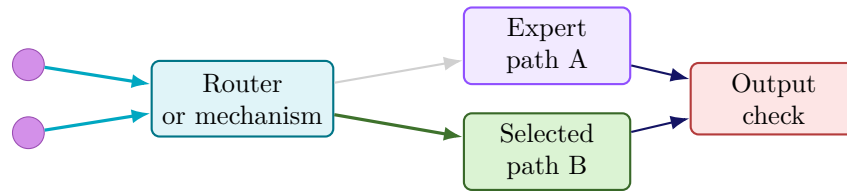
    Planned purpose: FeedForward module with Linear, activation, Linear.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Confirm FFN preserves the final D dimension.

Attention and FFN combine inside a residual decoder block.



Ch. 2.4: Dense feed-forward block as the replacement target

Figure 2.4: Dense FFN applied independently to each token row.

Table 2.4: Dense FFN parameters compared with one future expert MLP.

Item	Planned content	Status
Mechanism	Implement the dense FFN in a way that makes the later expert replacement obvious.	Placeholder
Mini-example	Apply the same MLP to every token in a B*T flattened view.	Placeholder
Diagnostic	Confirm FFN preserves the final D dimension.	Placeholder

2.5 Training loop and sampling baseline

Train the dense model long enough to create a trustworthy comparison anchor.

Run a smoke train for a few iterations and sample a short continuation.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Cross-entropy over shifted logits and targets.} \quad (2.5)$$

Listing 2.5: Planned code placeholder for training loop and sampling baseline.

```

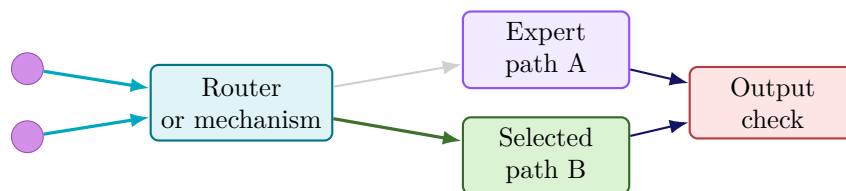
def training_loop_and_sampling_baseline_placeholder(x, config):
    """Chapter 2.5 placeholder.

    Planned purpose: Train step with loss, backward pass, clipping, optimizer step, and logging.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
  
```

Checkpoint

Track train and validation loss plus samples at fixed intervals.

Now that the dense baseline works, the next chapter isolates the expert and router pieces.



Ch. 2.5: Training loop and sampling baseline

Figure 2.5: Training loop pipeline: batch, forward, loss, backward, optimizer, sample.

Table 2.5: Baseline configuration fields and default smoke-test values.

Item	Planned content	Status
Mechanism	Train the dense model long enough to create a trustworthy comparison anchor.	Placeholder
Mini-example	Run a smoke train for a few iterations and sample a short continuation.	Placeholder
Diagnostic	Track train and validation loss plus samples at fixed intervals.	Placeholder

2.6 Baseline diagnostics and comparison slots

Prepare diagnostic hooks that will later compare dense and sparse models fairly.

Create a metrics dictionary with loss, tokens/sec, parameter count, and activation estimate.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Perplexity} = \exp(\text{validation cross-entropy}). \quad (2.6)$$

Listing 2.6: Planned code placeholder for baseline diagnostics and comparison slots.

```

def baseline_diagnostics_and_comparison_slots_placeholder(x, config):
    """Chapter 2.6 placeholder.

    Planned purpose: count_parameters and estimate_active_parameters helper functions.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Write the first baseline row in the experiment comparison table.

The next chapter replaces the dense FFN with a small expert bank and a router.

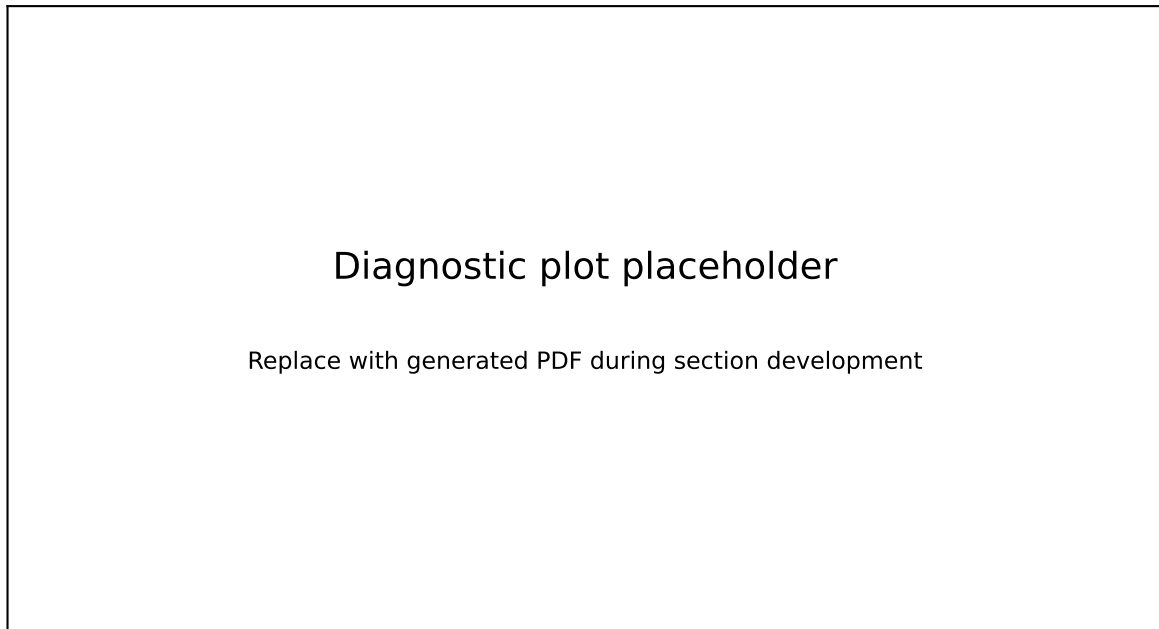


Figure 2.6: Dashboard placeholder for loss curves, parameter counts, and future routing metrics.

Table 2.6: Metrics available now versus metrics added by MoE layers.

Item	Planned content	Status
Mechanism	Prepare diagnostic hooks that will later compare dense and sparse models fairly.	Placeholder
Mini-example	Create a metrics dictionary with loss, tokens/sec, parameter count, and activation estimate.	Placeholder
Diagnostic	Write the first baseline row in the experiment comparison table.	Placeholder

Chapter 3

Experts, routers, and the first MoE layer

This chapter covers

- How an expert is just an FFN with an expert identity.
- How a router turns token states into expert scores.
- How a naive loop implementation clarifies dispatch before optimization.

Introduce the MoE layer as a transparent replacement for the dense FFN.

3.1 Expert MLPs as named FFNs

Demystify experts by deriving them directly from the dense FFN module.

Clone the dense FFN four times and label the copies expert 0 through expert 3.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{expert-}i(x) \text{ maps } \mathbb{R}^D \text{ to } \mathbb{R}^D \text{ for every expert } i. \quad (3.1)$$

Listing 3.1: Planned code placeholder for expert MLPs as named FFNs.

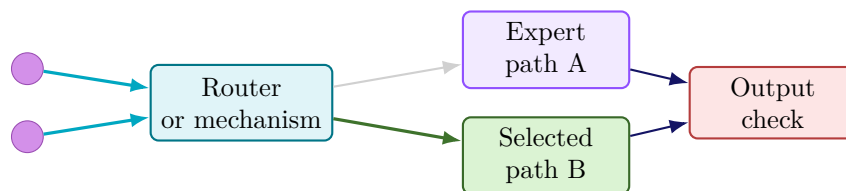
```
def expert_mlp_as_named_ffn_placeholder(x, config):
    """Chapter 3.1 placeholder.

    Planned purpose: ExpertMLP class with the same input-output contract as the dense FFN.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Run all experts on the same token and compare output shapes.

Experts need a token-dependent selection mechanism: the router.



Ch. 3.1: Expert MLPs as named FFNs

Figure 3.1: One dense FFN replaced by a bank of same-shape expert MLPs.

Table 3.1: Dense FFN versus expert bank parameters, active parameters, and outputs.

Item	Planned content	Status
Mechanism	Demystify experts by deriving them directly from the dense FFN module.	Placeholder
Mini-example	Clone the dense FFN four times and label the copies expert 0 through expert 3.	Placeholder
Diagnostic	Run all experts on the same token and compare output shapes.	Placeholder

3.2 Router scores and tensor shapes

Define router logits, router probabilities, and the flattening convention used throughout the book.

Flatten x from (B,T,D) to (N,D) and compute `router_logits` with $E=4$ experts.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{router-logits} = x\text{-flat } W\text{-r} + b\text{-r.} \quad (3.2)$$

Listing 3.2: Planned code placeholder for router scores and tensor shapes.

```

def router_scores_and_tensor_shapes_placeholder(x, config):
    """Chapter 3.2 placeholder.

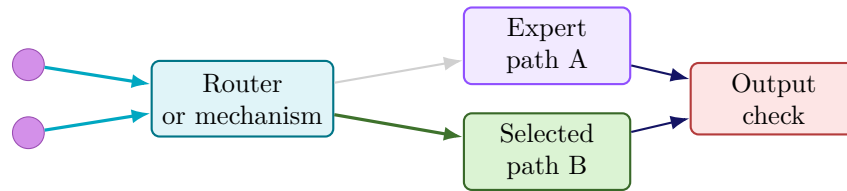
    Planned purpose: Router module that returns logits and score tensors.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Assert that the expert dimension equals `n_experts` and token dimension equals $B \cdot T$.

Scores become sparse when we select only the top experts per token.



Ch. 3.2: Router scores and tensor shapes

Figure 3.2: Flattened token matrix multiplied by a router projection to produce an N by E score table.

Table 3.2: Router tensor shape contract for x, x-flat, logits, scores, topk-idx, and gates.

Item	Planned content	Status
Mechanism	Define router logits, router probabilities, and the flattening convention used throughout the book.	Placeholder
Mini-example	Flatten x from (B,T,D) to (N,D) and compute router-logits with E=4 experts.	Placeholder
Diagnostic	Assert that the expert dimension equals n-experts and token dimension equals B*T.	Placeholder

3.3 Top-k selection without dispatch

Separate the selection problem from expert execution so readers can inspect router behavior first.

Compute top-2 expert IDs and scores for a four-token, four-expert score table.

The planned mechanics should introduce the equation before the implementation listing.

$$S_t = \text{topk}(\text{score}_t, K), \text{ one selected set per token.} \quad (3.3)$$

Listing 3.3: Planned code placeholder for top k selection without dispatch.

```

def top_k_selection_without_dispatch_placeholder(x, config):
    """Chapter 3.3 placeholder.

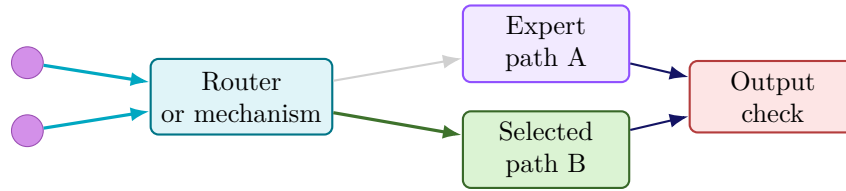
    Planned purpose: torch.topk example that returns topk_score and topk_idx.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Verify that every token has exactly K selected expert IDs.

Selection tells us where tokens should go; dispatch actually sends them there.



Ch. 3.3: Top-k selection without dispatch

Figure 3.3: Score matrix with the top-k entries highlighted in each row.

Table 3.3: Manual top-k results for the toy score table.

Item	Planned content	Status
Mechanism	Separate the selection problem from expert execution so readers can inspect router behavior first.	Placeholder
Mini-example	Compute top-2 expert IDs and scores for a four-token, four-expert score table.	Placeholder
Diagnostic	Verify that every token has exactly K selected expert IDs.	Placeholder

3.4 Naive token-by-token dispatch

Implement dispatch with loops first so the control flow is unmistakable.

For each token and selected expert, call the expert and add gate-weighted output.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{routed-out-t} = \sum_{j=1}^K \text{gate-}\{t_j\} \text{ expert-}\{\text{idx-}\{t_j\}\} (x-t). \quad (3.4)$$

Listing 3.4: Planned code placeholder for naive token by token dispatch.

```

def naive_token_by_token_dispatch_placeholder(x, config):
    """Chapter 3.4 placeholder.

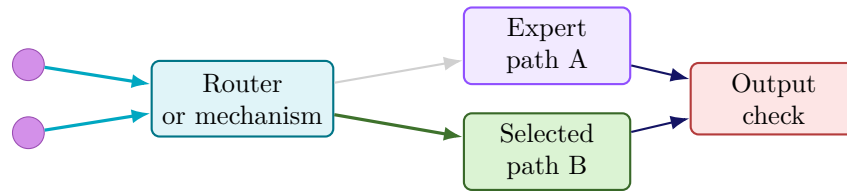
    Planned purpose: NaiveMoELayer forward pass using explicit token and expert loops.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Compare output shape to dense FFN output and inspect gates per token.

The naive layer works, but the router choices are still fragile; top-1 routing exposes this clearly.



Ch. 3.4: Naive token-by-token dispatch

Figure 3.4: Loop-based dispatch arrows from token rows to selected expert calls.

Table 3.4: Naive dispatch variables and their role in the loop.

Item	Planned content	Status
Mechanism	Implement dispatch with loops first so the control flow is unmistakable.	Placeholder
Mini-example	For each token and selected expert, call the expert and add gate-weighted output.	Placeholder
Diagnostic	Compare output shape to dense FFN output and inspect gates per token.	Placeholder

3.5 Shape and gradient checks

Make the first MoE layer testable before introducing more routing variants.

Run a two-layer smoke model and confirm all experts receive gradients when selected.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Loss gradient reaches gate and expert paths through the weighted sum.} \quad (3.5)$$

Listing 3.5: Planned code placeholder for shape and gradient checks.

```

def shape_and_gradient_checks_placeholder(x, config):
    """Chapter 3.5 placeholder.

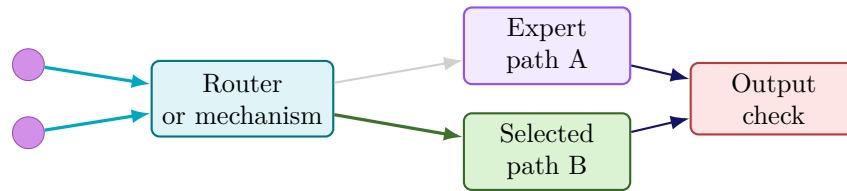
    Planned purpose: Unit tests for MoELayer output shape and gradient presence.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Count selected expert IDs and check nonzero gradients for selected expert parameters.

The next chapter simplifies to top-1 routing to teach sparse dispatch one expert at a time.



Ch. 3.5: Shape and gradient checks

Figure 3.5: Gradient flow diagram from loss through combine weights into router and selected experts.

Table 3.5: Test cases for shapes, gate sums, selected expert counts, and gradients.

Item	Planned content	Status
Mechanism	Make the first MoE layer testable before introducing more routing variants.	Placeholder
Mini-example	Run a two-layer smoke model and confirm all experts receive gradients when selected.	Placeholder
Diagnostic	Count selected expert IDs and check nonzero gradients for selected expert parameters.	Placeholder

3.6 Chapter summary and handoff

Summarize the first complete MoE layer and prepare for routing variants.

Re-run the four-token example through ExpertMLP, Router, top-k, dispatch, and combine.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{MoE layer invariant } y \text{ has the same shape as } x. \quad (3.6)$$

Listing 3.6: Planned code placeholder for chapter summary and handoff.

```

def chapter_summary_and_handoff_placeholder(x, config):
    """Chapter 3.6 placeholder.

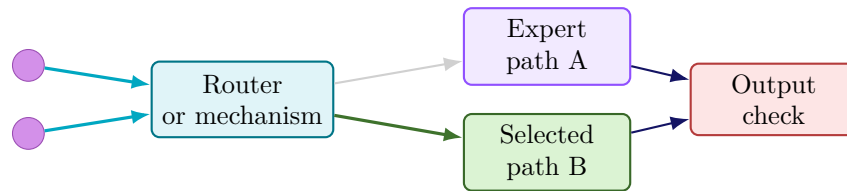
    Planned purpose: Minimal integration snippet replacing DenseFFN with NaiveMoELayer.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Reader checkpoint: identify where routing, expert computation, and combine happen in code.

Top-1 routing is the cleanest way to see sparse expert assignment and load imbalance.



Ch. 3.6: Chapter summary and handoff

Figure 3.6: End-to-end first MoE layer pipeline.

Table 3.6: Completed components and known limitations.

Item	Planned content	Status
Mechanism	Summarize the first complete MoE layer and prepare for routing variants.	Placeholder
Mini-example	Re-run the four-token example through ExpertMLP, Router, top-k, dispatch, and combine.	Placeholder
Diagnostic	Reader checkpoint: identify where routing, expert computation, and combine happen in code.	Placeholder

Chapter 4

Top-1 routing and Switch-style dispatch

This chapter covers

- How top-1 routing sends each token to exactly one expert.
- How capacity and dropped tokens appear when experts are overloaded.
- How expert-load diagnostics make routing failures visible.

Build the simplest sparse router and expose the expert-load problem.

4.1 Why top-1 routing is the simplest sparse case

Use one-expert-per-token routing to reduce the dispatch problem to assignment plus combine.

Route six tokens to four experts with $K=1$ and mark each token assignment.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{expert-idx-t} = \text{argmax-i score-}\{t,i\}. \quad (4.1)$$

Listing 4.1: Planned code placeholder for why top 1 routing is the simplest sparse case.

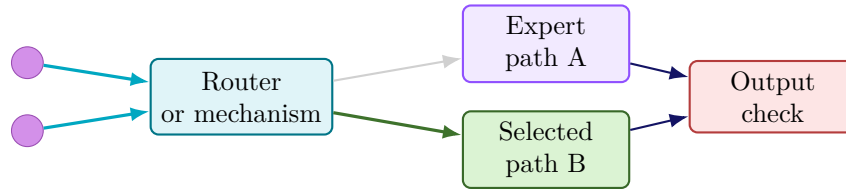
```
def why_top_1_routing_is_the_simplest_sparse_case_placeholder(x, config):
    """Chapter 4.1 placeholder.

    Planned purpose: Top1Router forward pass returning expert_idx and gate.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Verify each token has one expert and one scalar gate.

The single expert ID can be converted into an assignment mask.



Ch. 4.1: Why top-1 routing is the simplest sparse case

Figure 4.1: Token rows each connected to one selected expert.

Table 4.1: Top-1 routing variables compared with top-k routing variables.

Item	Planned content	Status
Mechanism	Use one-expert-per-token routing to reduce the dispatch problem to assignment plus combine.	Placeholder
Mini-example	Route six tokens to four experts with K=1 and mark each token assignment.	Placeholder
Diagnostic	Verify each token has one expert and one scalar gate.	Placeholder

4.2 Assignment masks and expert buckets

Convert token expert IDs into per-expert token groups.

Bucket six token indices into four expert lists.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{mask-}\{t,i\} = 1 \text{ when expert-idx-}t = i. \quad (4.2)$$

Listing 4.2: Planned code placeholder for assignment masks and expert buckets.

```

def assignment_masks_and_expert_buckets_placeholder(x, config):
    """Chapter 4.2 placeholder.

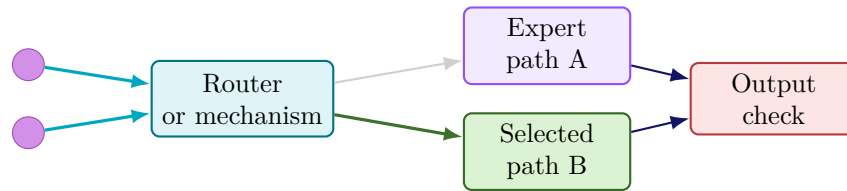
    Planned purpose: Build expert_to_token_indices from top-1 expert IDs.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Sum the mask across experts to confirm each token is assigned once.

Once tokens are bucketed, expert modules process only their assigned tokens.



Ch. 4.2: Assignment masks and expert buckets

Figure 4.2: Assignment mask as a sparse N by E matrix and as expert buckets.

Table 4.2: Mask representation versus bucket representation trade-offs.

Item	Planned content	Status
Mechanism	Convert token expert IDs into per-expert token groups.	Placeholder
Mini-example	Bucket six token indices into four expert lists.	Placeholder
Diagnostic	Sum the mask across experts to confirm each token is assigned once.	Placeholder

4.3 Dispatching tokens to selected experts

Run each expert only on the tokens assigned to it and scatter outputs back to token order.

Expert 0 receives token rows [0,3], expert 1 receives [1], and empty experts are skipped.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{output}[\text{token-indices-}i] = \text{expert-}i(\text{x}[\text{token-indices-}i]). \quad (4.3)$$

Listing 4.3: Planned code placeholder for dispatching tokens to selected experts.

```

def dispatching_tokens_to_selected_experts_placeholder(x, config):
    """Chapter 4.3 placeholder.

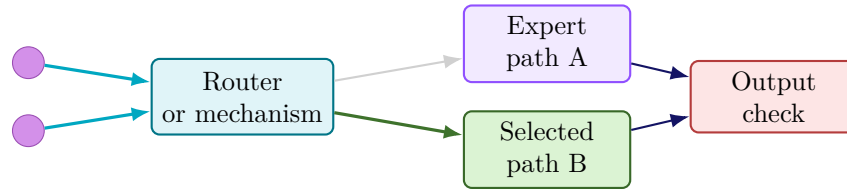
    Planned purpose: Top1MoELayer dispatch and scatter implementation.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Assert output rows return to the original token order.

Routing can overload an expert, so we need to define expert capacity.



Ch. 4.3: Dispatching tokens to selected experts

Figure 4.3: Gather tokens into expert batches, process, then scatter back.

Table 4.3: Gather, expert batch, and scatter tensor shapes.

Item	Planned content	Status
Mechanism	Run each expert only on the tokens assigned to it and scatter outputs back to token order.	Placeholder
Mini-example	Expert 0 receives token rows [0,3], expert 1 receives [1], and empty experts are skipped.	Placeholder
Diagnostic	Assert output rows return to the original token order.	Placeholder

4.4 Capacity and dropped tokens

Introduce the capacity factor and the consequences of too many tokens choosing the same expert.

Set capacity=2 for four experts and show what happens when four tokens select expert 0.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{capacity} = \text{ceil}(\text{capacity-factor} * N / E). \quad (4.4)$$

Listing 4.4: Planned code placeholder for capacity and dropped tokens.

```

def capacity_and_dropped_tokens_placeholder(x, config):
    """Chapter 4.4 placeholder.

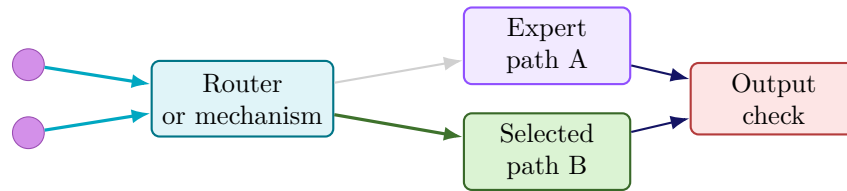
    Planned purpose: Capacity-aware top-1 dispatch with dropped-token accounting.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Report `dropped_token_count` and per-expert accepted counts.

Capacity exposes routing imbalance; histograms make the imbalance easy to see.



Ch. 4.4: Capacity and dropped tokens

Figure 4.4: Overloaded expert bucket with accepted and dropped token slots.

Table 4.4: Capacity factor, expert capacity, accepted tokens, dropped tokens, and residual fallback.

Item	Planned content	Status
Mechanism	Introduce the capacity factor and the consequences of too many tokens choosing the same expert.	Placeholder
Mini-example	Set capacity=2 for four experts and show what happens when four tokens select expert 0.	Placeholder
Diagnostic	Report dropped-token-count and per-expert accepted counts.	Placeholder

4.5 Load histograms and failure modes

Make expert collapse and underuse visible with simple diagnostics.

Create a synthetic router that sends most tokens to one expert and plot the load.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{load-fraction-i} = \text{count-i} / N. \quad (4.5)$$

Listing 4.5: Planned code placeholder for load histograms and failure modes.

```

def load_histograms_and_failure_modes_placeholder(x, config):
    """Chapter 4.5 placeholder.

    Planned purpose: Function that computes expert_load, load_fraction, and routing_entropy.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Save a histogram and a warning if max load is much larger than average load.

Top-1 is simple but brittle; top-2 routing gives each token a backup path and a weighted combine.

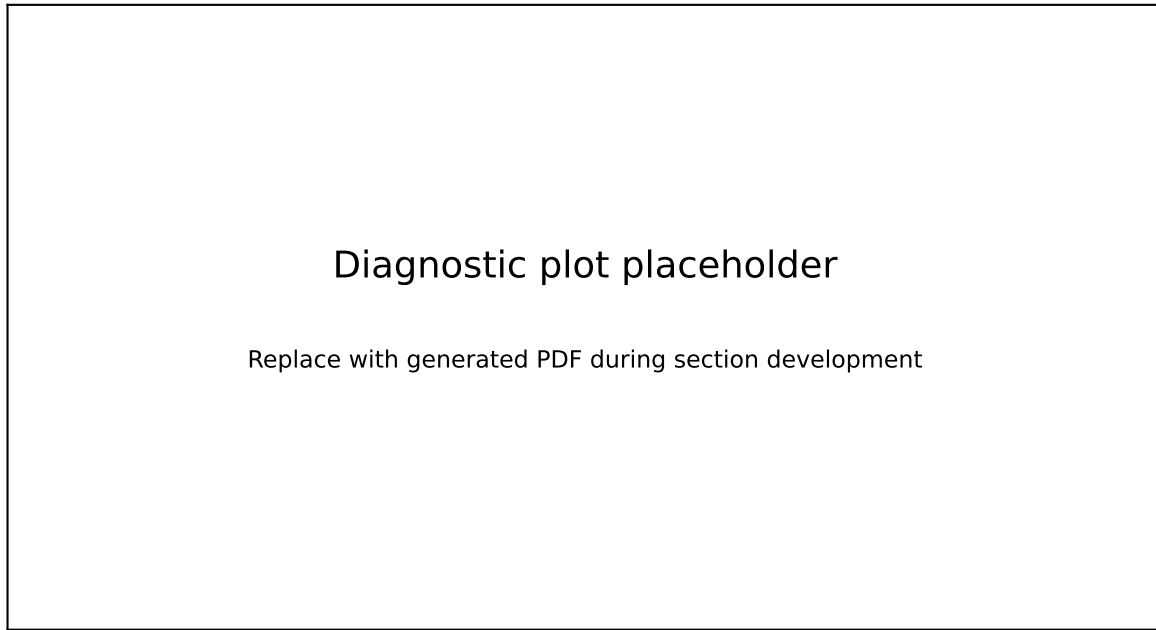


Figure 4.5: Expert-load histogram with one overloaded expert highlighted in imbalance red.

Table 4.5: Failure modes: expert collapse, empty experts, high drop rate, low entropy.

Item	Planned content	Status
Mechanism	Make expert collapse and underuse visible with simple diagnostics.	Placeholder
Mini-example	Create a synthetic router that sends most tokens to one expert and plot the load.	Placeholder
Diagnostic	Save a histogram and a warning if max load is much larger than average load.	Placeholder

4.6 Chapter summary and handoff

Consolidate top-1 routing as a working sparse layer and name its limitations.

Run the top-1 layer inside the dense decoder block and log load statistics.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Top-1 output as one gated expert output per token.} \quad (4.6)$$

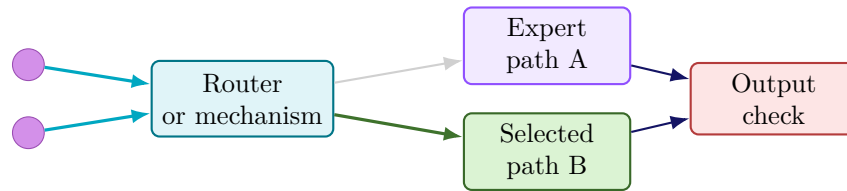
Listing 4.6: Planned code placeholder for chapter summary and handoff.

```
def chapter_summary_and_handoff_placeholder(x, config):
    """Chapter 4.6 placeholder.

    Planned purpose: Replace DenseFFN with Top1MoELayer in the baseline block.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Compare validation loss smoke run with dense baseline, without claiming quality superiority.



Ch. 4.6: Chapter summary and handoff

Figure 4.6: Top-1 MoE block inside the Transformer decoder layer.

Table 4.6: What top-1 solved versus what remains for top-2 routing.

Item	Planned content	Status
Mechanism	Consolidate top-1 routing as a working sparse layer and name its limitations.	Placeholder
Mini-example	Run the top-1 layer inside the dense decoder block and log load statistics.	Placeholder
Diagnostic	Compare validation loss smoke run with dense baseline, without claiming quality superiority.	Placeholder

The next chapter generalizes the same layer to top-2 weighted expert combinations.

Chapter 5

Top-2 routing and weighted expert combination

This chapter covers

- How top-2 routing keeps sparse compute while reducing one-expert brittleness.
- How selected scores become normalized gates for weighted expert outputs.
- How to compare top-1 and top-2 layers with the same diagnostics.

Generalize sparse routing so each token can combine two expert transformations.

5.1 Why top-2 changes the routing story

Explain top-2 routing as sparse ensemble behavior at the token level.

Use four tokens and four experts; each token chooses two experts with different weights.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Each selected set } S\text{-}t \text{ contains } K=2 \text{ expert IDs.} \quad (5.1)$$

Listing 5.1: Planned code placeholder for why top 2 changes the routing story.

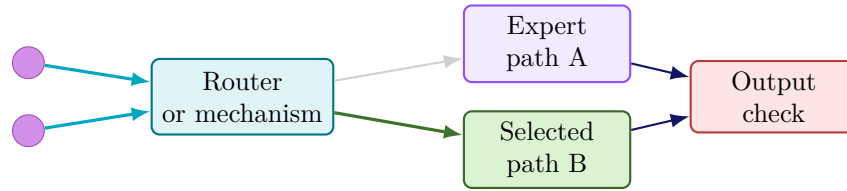
```
def why_top_2_changes_the_routing_story_placeholder(x, config):
    """Chapter 5.1 placeholder.

    Planned purpose: Config switch from K=1 to K=2 and resulting tensor shapes.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Compare active expert calls per token for K=1 and K=2.

To make top-2 work, selected scores must become stable combine weights.



Ch. 5.1: Why top-2 changes the routing story

Figure 5.1: Token routes split into two weighted paths.

Table 5.1: Top-1 versus top-2 routing behavior, cost, and diagnostics.

Item	Planned content	Status
Mechanism	Explain top-2 routing as sparse ensemble behavior at the token level.	Placeholder
Mini-example	Use four tokens and four experts; each token chooses two experts with different weights.	Placeholder
Diagnostic	Compare active expert calls per token for K=1 and K=2.	Placeholder

5.2 Normalizing selected router scores

Turn top-2 scores into gates that sum to one for each token.

Normalize selected scores [0.8, 0.2] and [0.51, 0.49] and compare combines.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{gate-}\{t_j\} = \text{score-}\{t, \text{idx-}\{t_j\}\} / \text{sum-l score-}\{t, \text{idx-}\{t_l\}\}. \quad (5.2)$$

Listing 5.2: Planned code placeholder for normalizing selected router scores.

```

def normalizing_selected_router_scores_placeholder(x, config):
    """Chapter 5.2 placeholder.

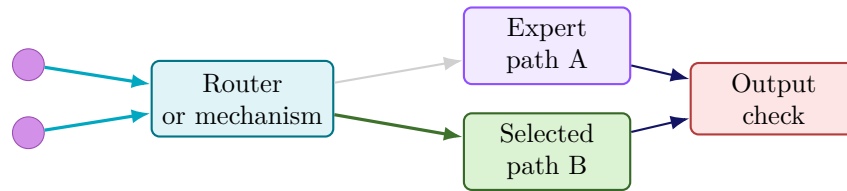
    Planned purpose: selected_scores / selected_scores.sum(dim=-1, keepdim=True).
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Assert `torch.allclose(gates.sum(-1), ones)`.

The normalized gates weight two expert outputs per token.



Ch. 5.2: Normalizing selected router scores

Figure 5.2: Top-2 score row transformed into a two-weight gate vector.

Table 5.2: Raw scores, selected scores, normalized gates, and gate sums.

Item	Planned content	Status
Mechanism	Turn top-2 scores into gates that sum to one for each token.	Placeholder
Mini-example	Normalize selected scores [0.8, 0.2] and [0.51, 0.49] and compare combines.	Placeholder
Diagnostic	Assert torch.allclose(gates.sum(-1), ones).	Placeholder

5.3 Dispatch for two expert paths per token

Adapt the dispatch loop so each selected expert path contributes to the same output row.

Token 0 goes to experts 1 and 3; both outputs are accumulated into output[0].

The planned mechanics should introduce the equation before the implementation listing.

$$\text{out-t} += \text{gate-}\{tj\} \text{ expert-}\{idx-\{tj\}\}(x-t). \quad (5.3)$$

Listing 5.3: Planned code placeholder for dispatch for two expert paths per token.

```

def dispatch_for_two_expert_paths_per_token_placeholder(x, config):
    """Chapter 5.3 placeholder.

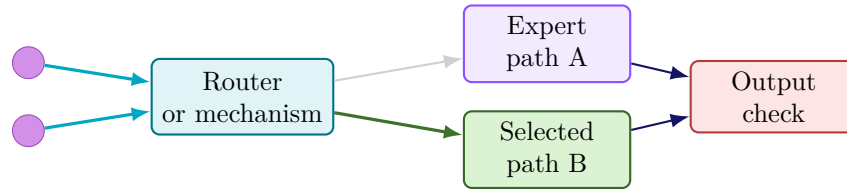
    Planned purpose: Top2MoELayer forward pass with nested selected-expert accumulation.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Check output changes when gates are manually swapped.

Top-2 dispatch is correct but still slow; vectorization comes after the routing math is clear.



Ch. 5.3: Dispatch for two expert paths per token

Figure 5.3: Two paths per token gathered into expert batches and accumulated back.

Table 5.3: Data structures for topk-idx, gates, expert batches, and output accumulation.

Item	Planned content	Status
Mechanism	Adapt the dispatch loop so each selected expert path contributes to the same output row.	Placeholder
Mini-example	Token 0 goes to experts 1 and 3; both outputs are accumulated into output[0].	Placeholder
Diagnostic	Check output changes when gates are manually swapped.	Placeholder

5.4 Comparing top-1 and top-2 diagnostics

Use the same metrics to show how K changes load, drop rate, and routing entropy.

Run the same random batch through top-1 and top-2 routers with the same scores.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Active expert calls equal } N \cdot K \text{ before capacity limits.} \quad (5.4)$$

Listing 5.4: Planned code placeholder for comparing top 1 and top 2 diagnostics.

```

def comparing_top_1_and_top_2_diagnostics_placeholder(x, config):
    """Chapter 5.4 placeholder.

    Planned purpose: compare_routing_modes(router_scores, k_values=[1,2]).
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Save a comparison table and one histogram per routing mode.

Once routing variants share diagnostics, the next challenge is making dispatch less loop-heavy.

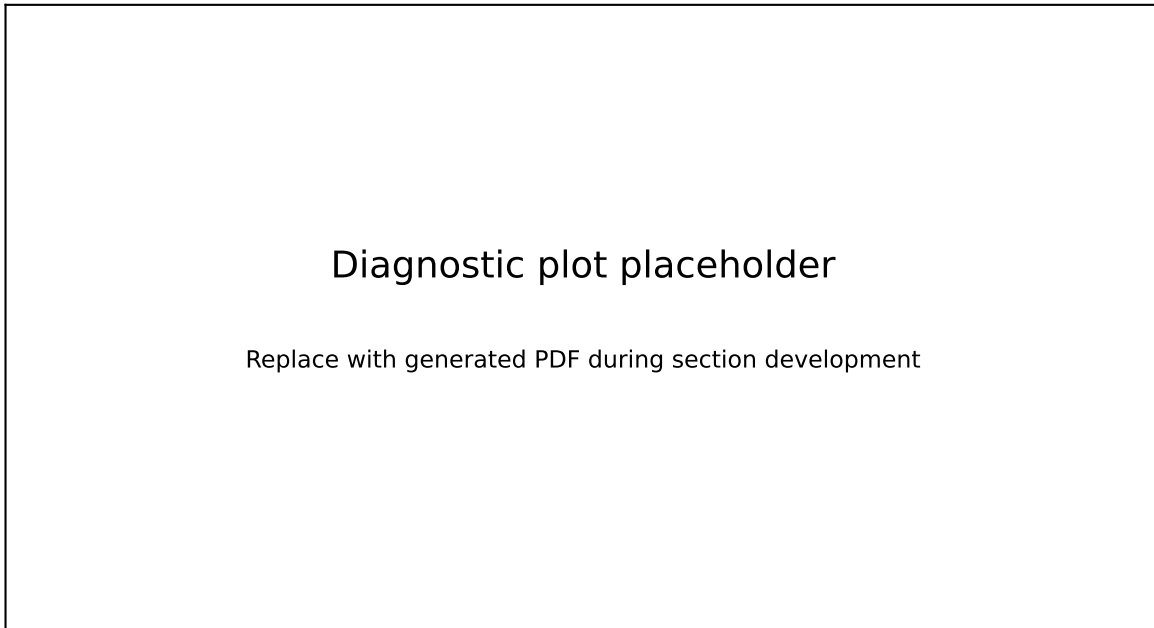


Figure 5.4: Side-by-side expert load histograms for K=1 and K=2.

Table 5.4: Comparison rows for active calls, max load, entropy, and gate concentration.

Item	Planned content	Status
Mechanism	Use the same metrics to show how K changes load, drop rate, and routing entropy.	Placeholder
Mini-example	Run the same random batch through top-1 and top-2 routers with the same scores.	Placeholder
Diagnostic	Save a comparison table and one histogram per routing mode.	Placeholder

5.5 Numerical stability in top-k gates

Prevent gate normalization edge cases before they become training bugs.

Use tiny selected scores and demonstrate epsilon-safe normalization.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{gate} = \text{selected-scores} / \text{clamp}(\text{sum}(\text{selected-scores}), \text{eps}). \quad (5.5)$$

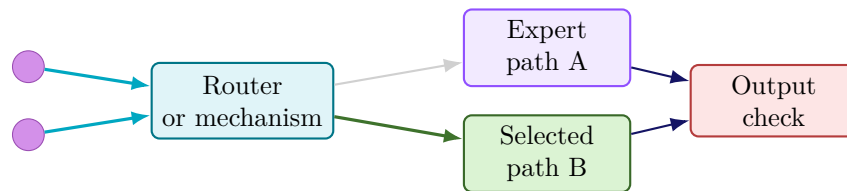
Listing 5.5: Planned code placeholder for numerical stability in top k gates.

```
def numerical_stability_in_top_k_gates_placeholder(x, config):
    """Chapter 5.5 placeholder.

    Planned purpose: stable_normalize_selected_scores function with epsilon and dtype checks.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Unit tests for finite gates and gradients under small scores.



Ch. 5.5: Numerical stability in top-k gates

Figure 5.5: Gate normalization failure and epsilon-stabilized path.

Table 5.5: Potential failures: zero sums, dtype mismatch, overflow, and unintended detached gates.

Item	Planned content	Status
Mechanism	Prevent gate normalization edge cases before they become training bugs.	Placeholder
Mini-example	Use tiny selected scores and demonstrate epsilon-safe normalization.	Placeholder
Diagnostic	Unit tests for finite gates and gradients under small scores.	Placeholder

Now that top-2 is correct, we can rewrite the dispatch to scale beyond tiny examples.

5.6 Chapter summary and handoff

Lock in the top-2 routing contract used by MiniDeepSeekMoE.

Trace one token through score, top-2 indices, gates, two experts, and final output.

The planned mechanics should introduce the equation before the implementation listing.

MoE output as a normalized weighted sum over two experts. (5.6)

Listing 5.6: Planned code placeholder for chapter summary and handoff.

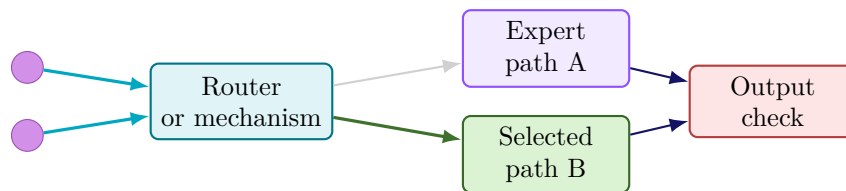
```
def chapter_summary_and_handoff_placeholder(x, config):
    """Chapter 5.6 placeholder.

    Planned purpose: Test fixture that instantiates K=2 MoELayer and checks shape plus gate sums.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Reader checkpoint: explain why gates sum to one after selecting top-k only.

The next chapter keeps the same math but changes the implementation to vectorized dispatch and capacity-aware batching.



Ch. 5.6: Chapter summary and handoff

Figure 5.6: Top-2 MoE layer end-to-end tensor map.

Table 5.6: Top-2 contracts that later chapters must preserve.

Item	Planned content	Status
Mechanism	Lock in the top-2 routing contract used by MiniDeepSeekMoE.	Placeholder
Mini-example	Trace one token through score, top-2 indices, gates, two experts, and final output.	Placeholder
Diagnostic	Reader checkpoint: explain why gates sum to one after selecting top-k only.	Placeholder

Chapter 6

Vectorized dispatch, capacity, and routing efficiency

This chapter covers

- How flattened tokens and expert batches make MoE dispatch vectorizable.
- How capacity limits create predictable tensor sizes and overflow behavior.
- How to test equivalence between naive and vectorized implementations.

Move from readable loops to scalable tensor operations without changing the MoE math.

6.1 Flattened tokens as the dispatch unit

Standardize on $N=B*T$ flattened tokens so routing code ignores batch layout until the final reshape.

Flatten $B=2$, $T=3$, $D=4$ into $N=6$ token rows and recover B,T,D afterward.

The planned mechanics should introduce the equation before the implementation listing.

$$n = b*T + t \text{ maps } (b,t) \text{ to flat token index.} \quad (6.1)$$

Listing 6.1: Planned code placeholder for flattened tokens as the dispatch unit.

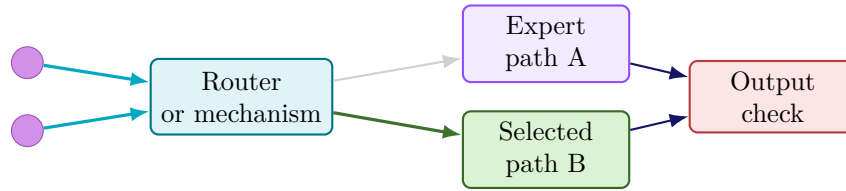
```
def flattened_tokens_as_the_dispatch_unit_placeholder(x, config):
    """Chapter 6.1 placeholder.

    Planned purpose: flatten_tokens and unflatten_tokens helper functions.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Round-trip flatten/unflatten equality check.

Flat token rows can be sorted or bucketed by expert assignment.



Ch. 6.1: Flattened tokens as the dispatch unit

Figure 6.1: Batch-sequence grid flattened into a token table for routing.

Table 6.1: Mapping between batch index, time index, flat index, and token vector.

Item	Planned content	Status
Mechanism	Standardize on $N=B*T$ flattened tokens so routing code ignores batch layout until the final reshape.	Placeholder
Mini-example	Flatten $B=2$, $T=3$, $D=4$ into $N=6$ token rows and recover B,T,D afterward.	Placeholder
Diagnostic	Round-trip flatten/unflatten equality check.	Placeholder

6.2 Expert batch construction

Build dense per-expert mini-batches from sparse token assignments.

Create an expert batch tensor with shape $(E, \text{capacity}, D)$.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{expert-batch}[i, \text{slot}, :] = \text{x-flat}[\text{token-idx}]. \quad (6.2)$$

Listing 6.2: Planned code placeholder for expert batch construction.

```

def expert_batch_construction_placeholder(x, config):
    """Chapter 6.2 placeholder.

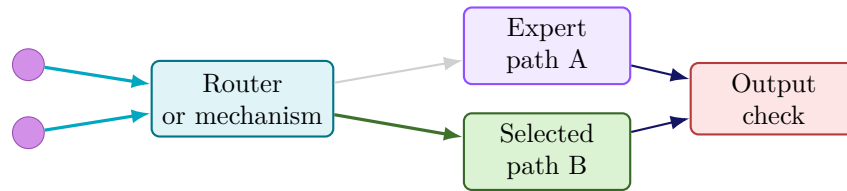
    Planned purpose: pack_tokens_by_expert function for top-k assignments.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Ensure each accepted route has exactly one expert slot.

Fixed-size expert batches require a capacity rule.



Ch. 6.2: Expert batch construction

Figure 6.2: Sparse token assignments packed into a fixed-size expert batch grid.

Table 6.2: expert-batch, combine-weights, token-indices, and slot-indices shapes.

Item	Planned content	Status
Mechanism	Build dense per-expert mini-batches from sparse token assignments.	Placeholder
Mini-example	Create an expert batch tensor with shape (E, capacity, D).	Placeholder
Diagnostic	Ensure each accepted route has exactly one expert slot.	Placeholder

6.3 Capacity factors and overflow policy

Explain how capacity controls memory and what the implementation does with overflow routes.

Use $N=12$, $E=4$, $K=2$ and capacity factors 1.0 and 1.25.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{capacity} = \text{ceil}(\text{capacity-factor} * N * K / E). \quad (6.3)$$

Listing 6.3: Planned code placeholder for capacity factors and overflow policy.

```

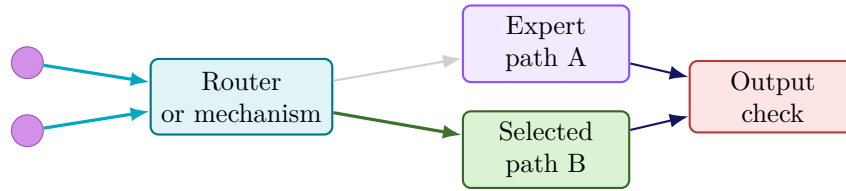
def capacity_factors_and_overflow_policy_placeholder(x, config):
    """Chapter 6.3 placeholder.

    Planned purpose: compute_expert_capacity and overflow mask code.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
  
```

Checkpoint

Log overflow route count and overflow fraction.

After expert batches are processed, outputs must be unpacked and combined.



Ch. 6.3: Capacity factors and overflow policy

Figure 6.3: Capacity slots per expert with accepted, padded, and overflow routes.

Table 6.3: Capacity factor scenarios and their accepted route counts.

Item	Planned content	Status
Mechanism	Explain how capacity controls memory and what the implementation does with overflow routes.	Placeholder
Mini-example	Use $N=12$, $E=4$, $K=2$ and capacity factors 1.0 and 1.25.	Placeholder
Diagnostic	Log overflow route count and overflow fraction.	Placeholder

6.4 Unpacking and weighted combine

Scatter expert-batch outputs back to token rows and apply route weights correctly.

One token receives two routes; one overflow route is skipped while the accepted route contributes.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{routed-out}[\text{token}] += \text{gate} * \text{expert-out}[\text{expert}, \text{slot}]. \quad (6.4)$$

Listing 6.4: Planned code placeholder for unpacking and weighted combine.

```

def unpacking_and_weighted_combine_placeholder(x, config):
    """Chapter 6.4 placeholder.

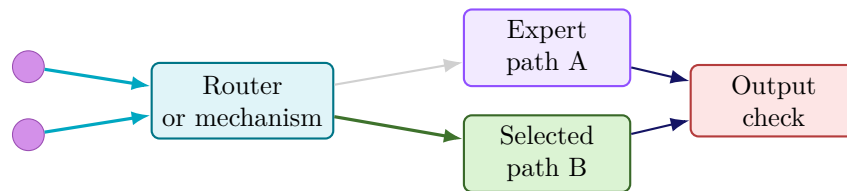
    Planned purpose: unpack_expert_outputs function using index_add or scatter_add.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Compare vectorized output to naive output when capacity is large enough for no overflow.

The implementation is faster, but correctness tests must guard against silent routing mistakes.



Ch. 6.4: Unpacking and weighted combine

Figure 6.4: Expert outputs unpacked through token and slot indices into routed-out.

Table 6.4: Accepted route table: token, expert, slot, gate, and output contribution.

Item	Planned content	Status
Mechanism	Scatter expert-batch outputs back to token rows and apply route weights correctly.	Placeholder
Mini-example	One token receives two routes; one overflow route is skipped while the accepted route contributes.	Placeholder
Diagnostic	Compare vectorized output to naive output when capacity is large enough for no overflow.	Placeholder

6.5 Equivalence tests against the naive layer

Prove the vectorized implementation matches the simple implementation in the no-overflow case.

Use fixed router scores and identical expert weights for both layers.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{max-abs-diff} = \max(|y\text{-naive} - y\text{-vec}|). \quad (6.5)$$

Listing 6.5: Planned code placeholder for equivalence tests against the naive layer.

```

def equivalence_tests_against_the_naive_layer_placeholder(x, config):
    """Chapter 6.5 placeholder.

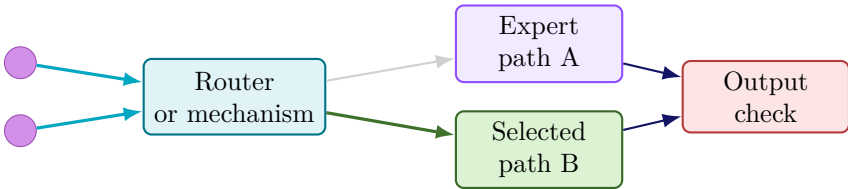
    Planned purpose: pytest-style allclose test comparing naive and vectorized MoE.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Fail if `max_abs_diff` exceeds tolerance in no-overflow settings.

With efficient dispatch available, we can focus on the training losses that shape router behavior.



Ch. 6.5: Equivalence tests against the naive layer

Figure 6.5: Two implementation paths producing matching output tensors.

Table 6.5: Test matrix for K, E, D, capacity factor, dtype, and overflow setting.

Item	Planned content	Status
Mechanism	Prove the vectorized implementation matches the simple implementation in the no-overflow case.	Placeholder
Mini-example	Use fixed router scores and identical expert weights for both layers.	Placeholder
Diagnostic	Fail if max-abs-diff exceeds tolerance in no-overflow settings.	Placeholder

6.6 Chapter summary and handoff

Summarize the efficient routing pipeline and the new capacity diagnostics.
Trace one batch through flatten, top-k, pack, expert compute, unpack, and reshape.
The planned mechanics should introduce the equation before the implementation listing.

No-overflow equivalence condition $y\text{-vec approx } y\text{-naive}.$

(6.6)

Listing 6.6: Planned code placeholder for chapter summary and handoff.

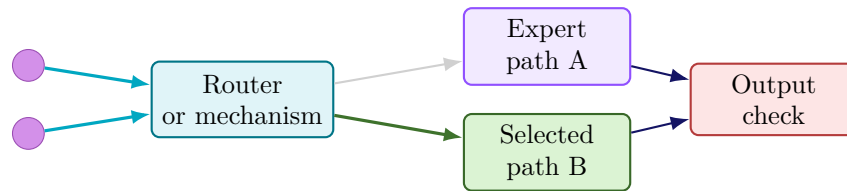
```
def chapter_summary_and_handoff_placeholder(x, config):
    """Chapter 6.6 placeholder.

    Planned purpose: VectorizedMoELayer forward skeleton with named helper calls.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Reader checkpoint: identify where dropped routes enter the output calculation.

The next chapter adds router training objectives so experts are used more evenly.



Ch. 6.6: Chapter summary and handoff

Figure 6.6: Full vectorized dispatch pipeline.

Table 6.6: Naive versus vectorized implementation responsibilities.

Item	Planned content	Status
Mechanism	Summarize the efficient routing pipeline and the new capacity diagnostics.	Placeholder
Mini-example	Trace one batch through flatten, top-k, pack, expert compute, unpack, and reshape.	Placeholder
Diagnostic	Reader checkpoint: identify where dropped routes enter the output calculation.	Placeholder

Chapter 7

Router training and load-balancing losses

This chapter covers

- Why next-token loss alone may not balance expert use.
- How auxiliary load-balancing losses and router regularizers are implemented.
- How routing diagnostics become part of the training loop.

Teach the router to avoid expert collapse before moving to auxiliary-loss-free balancing.

7.1 Expert imbalance as a training problem

Show that a technically correct router can still collapse onto a small number of experts.

Construct a score table where one expert receives most top-k selections.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Target average load per expert is } N \cdot K / E \text{ routes.} \quad (7.1)$$

Listing 7.1: Planned code placeholder for expert imbalance as a training problem.

```
def expert_imbalance_as_a_training_problem_placeholder(x, config):
    """Chapter 7.1 placeholder.

    Planned purpose: Synthetic imbalance generator for router-score diagnostics.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Report max_load/mean_load and number of empty experts.

An auxiliary loss can reward agreement between probability mass and actual expert load.

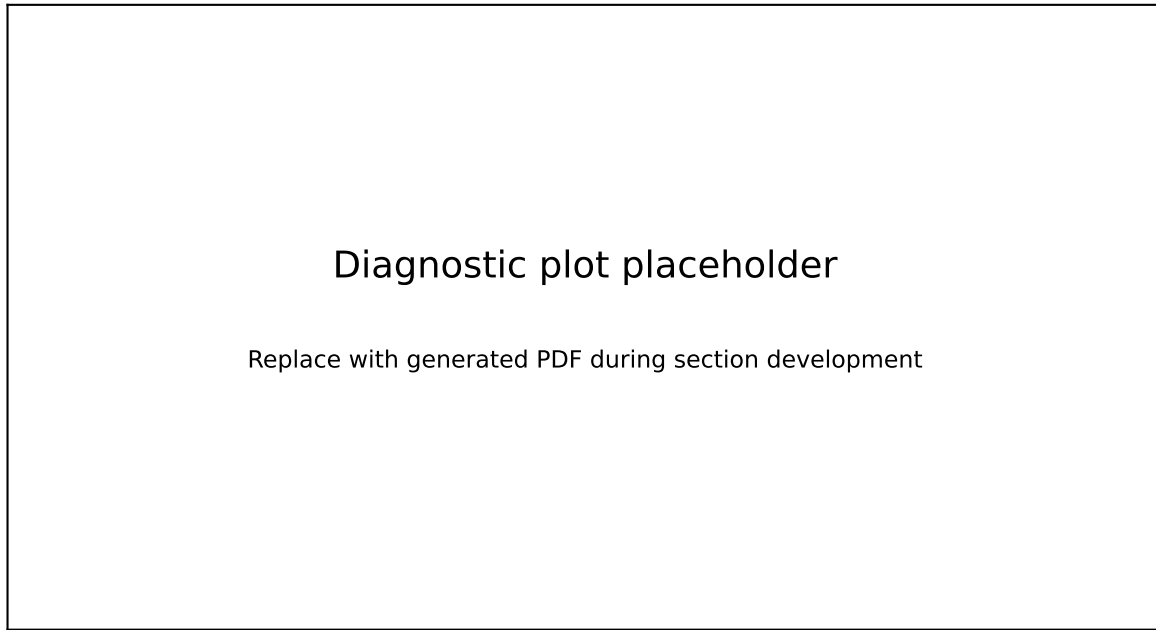


Figure 7.1: Collapsed routing histogram compared with balanced routing histogram.

Table 7.1: Symptoms of imbalance and the training behavior they affect.

Item	Planned content	Status
Mechanism	Show that a technically correct router can still collapse onto a small number of experts.	Placeholder
Mini-example	Construct a score table where one expert receives most top-k selections.	Placeholder
Diagnostic	Report max-load/mean-load and number of empty experts.	Placeholder

7.2 Auxiliary load-balancing loss

Implement the classic balancing baseline as a transparent, measurable loss term.

Compute expert load fractions and probability fractions for $E=4$ experts.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Auxiliary loss proportional to } E * \sum_i \text{load}_i * \text{prob}_i. \quad (7.2)$$

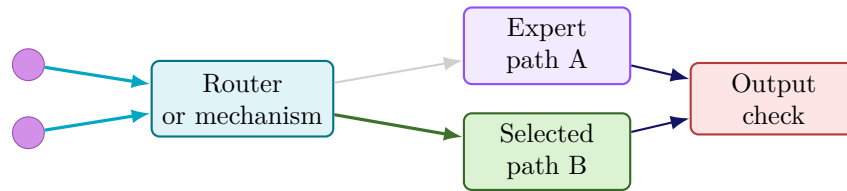
Listing 7.2: Planned code placeholder for auxiliary load balancing loss.

```
def auxiliary_load_balancing_loss_placeholder(x, config):
    """Chapter 7.2 placeholder.

    Planned purpose: load_balancing_loss function returning scalar aux_loss and metrics.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Check loss is lower for balanced synthetic routing than collapsed routing.



Ch. 7.2: Auxiliary load-balancing loss

Figure 7.2: Two bars per expert: selected load and router probability mass.

Table 7.2: Inputs and outputs of the auxiliary loss function.

Item	Planned content	Status
Mechanism	Implement the classic balancing baseline as a transparent, measurable loss term.	Placeholder
Mini-example	Compute expert load fractions and probability fractions for E=4 experts.	Placeholder
Diagnostic	Check loss is lower for balanced synthetic routing than collapsed routing.	Placeholder

Auxiliary loss enters the total training objective with a coefficient.

7.3 Integrating auxiliary loss into training

Add routing losses to the model output dictionary without hiding the main language-model loss.

Return `lm_loss`, `aux_loss`, `total_loss`, and `router_metrics` from one forward pass.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{total-loss} = \text{lm-loss} + \text{lambda-aux} * \text{aux-loss}. \quad (7.3)$$

Listing 7.3: Planned code placeholder for integrating auxiliary loss into training.

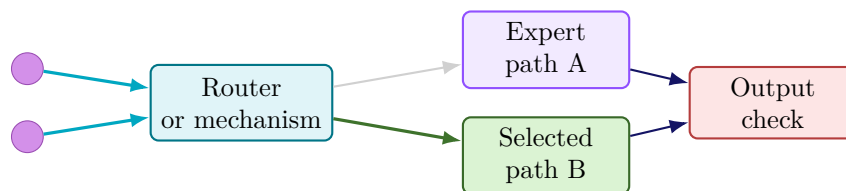
```
def integrating_auxiliary_loss_into_training_placeholder(x, config):
    """Chapter 7.3 placeholder.

    Planned purpose: Training step that combines lm_loss + aux_coef * aux_loss.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Log individual losses separately so balancing cannot hide LM degradation.

Load balancing is one router signal; score scale can also create instability.



Ch. 7.3: Integrating auxiliary loss into training

Figure 7.3: Loss composition diagram from logits and router statistics to total loss.

Table 7.3: Loss terms, coefficients, default values, and logging names.

Item	Planned content	Status
Mechanism	Add routing losses to the model output dictionary without hiding the main language-model loss.	Placeholder
Mini-example	Return lm-loss, aux-loss, total-loss, and router-metrics from one forward pass.	Placeholder
Diagnostic	Log individual losses separately so balancing cannot hide LM degradation.	Placeholder

7.4 Router score scale and z-loss

Introduce a simple regularizer for overly large router logits or scores.

Compare a modest logit row with a very large logit row that creates near-hard routing.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{z-loss based on squared logsumexp of router logits.} \quad (7.4)$$

Listing 7.4: Planned code placeholder for router score scale and z loss.

```

def router_score_scale_and_z_loss_placeholder(x, config):
    """Chapter 7.4 placeholder.

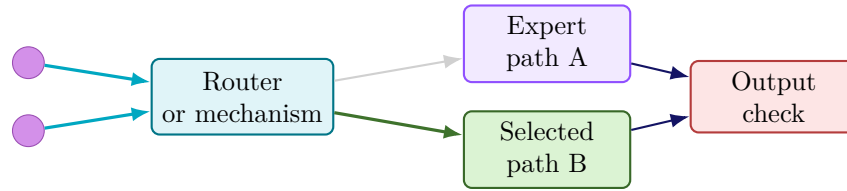
    Planned purpose: router_z_loss function and optional coefficient in the training config.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Track router score max, entropy, and z-loss over training.

The regularizers become useful only if the diagnostics are visible during training.



Ch. 7.4: Router score scale and z-loss

Figure 7.4: Router logit scale affects probability sharpness before top-k selection.

Table 7.4: Router regularization options: z-loss, entropy, noise, and clipping.

Item	Planned content	Status
Mechanism	Introduce a simple regularizer for overly large router logits or scores.	Placeholder
Mini-example	Compare a modest logit row with a very large logit row that creates near-hard routing.	Placeholder
Diagnostic	Track router score max, entropy, and z-loss over training.	Placeholder

7.5 Routing dashboards and training logs

Create the standard diagnostic views used for every later MoE experiment.

Log one training batch with load histogram, gate entropy, drop count, and loss terms.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Routing entropy over normalized selected gates.} \quad (7.5)$$

Listing 7.5: Planned code placeholder for routing dashboards and training logs.

```

def routing_dashboards_and_training_logs_placeholder(x, config):
    """Chapter 7.5 placeholder.

    Planned purpose: collect_router_metrics and plot_router_dashboard helpers.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Generate a dashboard PDF or image after a short smoke run.

The next chapter changes the expert architecture itself by adding shared and fine-grained routed experts.

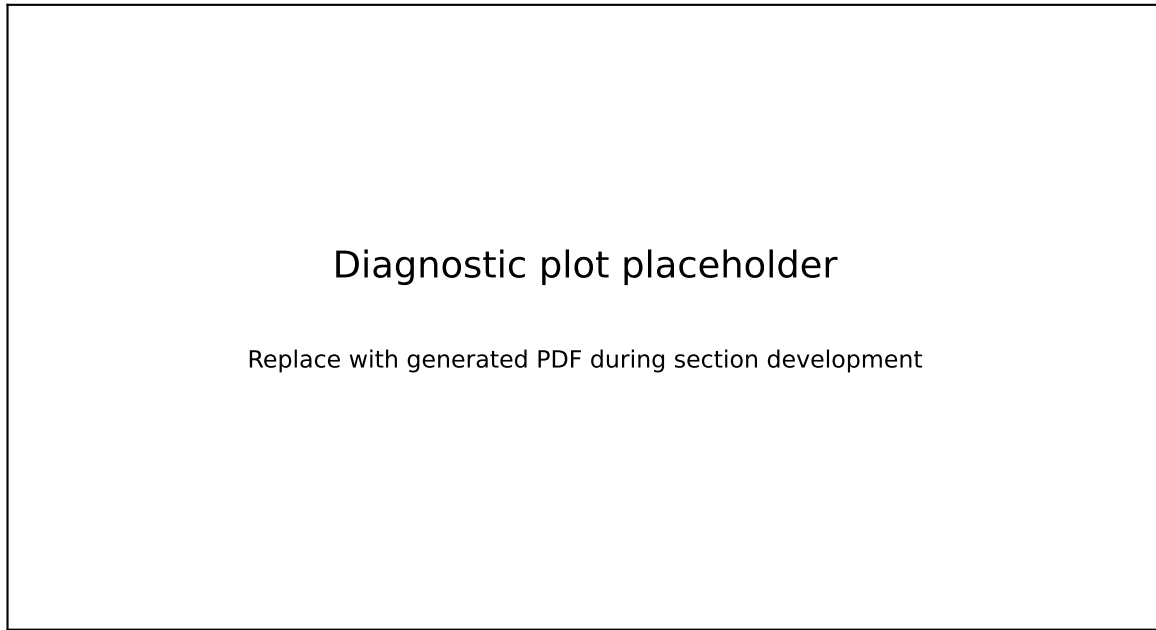


Figure 7.5: Dashboard layout with loss curve, expert load histogram, gate entropy, and drop rate.

Table 7.5: Metric names, tensor source, frequency, and expected range.

Item	Planned content	Status
Mechanism	Create the standard diagnostic views used for every later MoE experiment.	Placeholder
Mini-example	Log one training batch with load histogram, gate entropy, drop count, and loss terms.	Placeholder
Diagnostic	Generate a dashboard PDF or image after a short smoke run.	Placeholder

7.6 Chapter summary and handoff

Frame auxiliary balancing as the baseline that auxiliary-loss-free balancing will later improve on.

Compare training metrics with `aux_coef=0` and `aux_coef>0` in a smoke run.

The planned mechanics should introduce the equation before the implementation listing.

Model objective includes both LM and routing terms in this chapter. (7.6)

Listing 7.6: Planned code placeholder for chapter summary and handoff.

```
def chapter_summary_and_handoff_placeholder(x, config):
    """Chapter 7.6 placeholder.

    Planned purpose: Config presets for no-balancing and auxiliary-balancing runs.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Reader checkpoint: explain why `aux_loss` must be logged separately from LM loss.

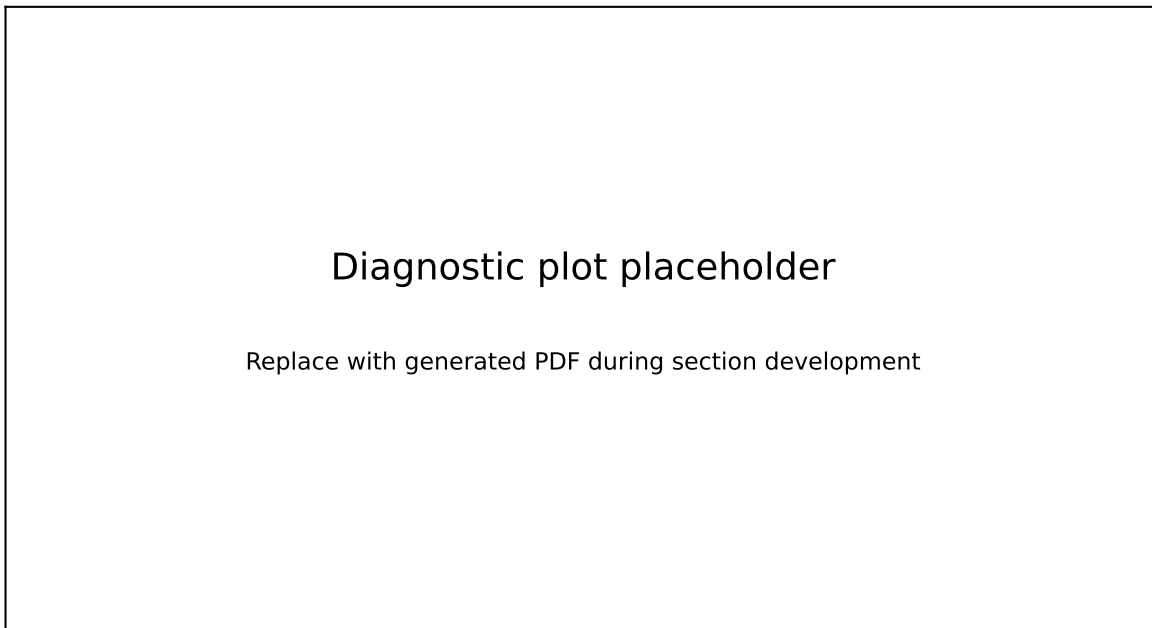


Figure 7.6: Before-and-after load histograms for auxiliary balancing.

Table 7.6: What auxiliary loss fixes and what trade-offs remain.

Item	Planned content	Status
Mechanism	Frame auxiliary balancing as the baseline that auxiliary-loss-free balancing will later improve on.	Placeholder
Mini-example	Compare training metrics with $\text{aux-coef}=0$ and $\text{aux-coef}>0$ in a smoke run.	Placeholder
Diagnostic	Reader checkpoint: explain why aux-loss must be logged separately from LM loss.	Placeholder

MiniDeepSeekMoE keeps the routing lessons but changes expert structure before changing the balancing method.

Chapter 8

Shared experts and fine-grained routed experts

This chapter covers

- Why shared experts provide an always-on path for common knowledge.
- Why fine-grained routed experts encourage more specialized sparse computation.
- How shared and routed expert outputs combine in one MoE block.

Implement the DeepSeekMoE-inspired expert structure for the reference model.

8.1 The role of shared experts

Explain the always-on expert path as a way to separate common processing from routed specialization.

Every token goes through one shared expert while also routing to two routed experts.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{shared-out} = \text{sum-s shared-expert-s}(x). \quad (8.1)$$

Listing 8.1: Planned code placeholder for the role of shared experts.

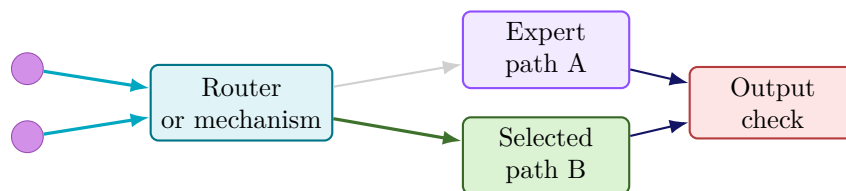
```
def the_role_of_shared_experts_placeholder(x, config):
    """Chapter 8.1 placeholder.

    Planned purpose: SharedExpertMLP module and shared_out calculation.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Verify shared experts receive gradients for every token, not only selected tokens.

Routed experts can also be made finer-grained to increase specialization options.



Ch. 8.1: The role of shared experts

Figure 8.1: Parallel shared path and routed path merging at the MoE output.

Table 8.1: Shared expert path versus routed expert path responsibilities.

Item	Planned content	Status
Mechanism	Explain the always-on expert path as a way to separate common processing from routed specialization.	Placeholder
Mini-example	Every token goes through one shared expert while also routing to two routed experts.	Placeholder
Diagnostic	Verify shared experts receive gradients for every token, not only selected tokens.	Placeholder

8.2 Fine-grained routed expert segmentation

Show how more smaller experts can replace fewer larger experts while preserving readable code.

Compare 4 large experts with `hidden_dim=1024` to 16 smaller experts with `hidden_dim=256`.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Expert-bank parameter estimate } E * (D * H - e + H - e * D). \quad (8.2)$$

Listing 8.2: Planned code placeholder for fine grained routed expert segmentation.

```

def fine_grained_routed_expert_segmentation_placeholder(x, config):
    """Chapter 8.2 placeholder.

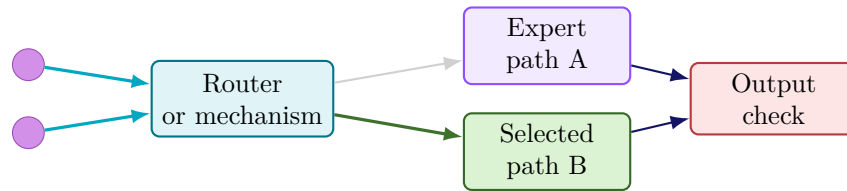
    Planned purpose: ExpertBank configuration that changes n_experts and expert_hidden_dim together.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Compute total and active expert parameters for several segmentations.

The reference layer combines fine-grained routed experts with shared experts.



Ch. 8.2: Fine-grained routed expert segmentation

Figure 8.2: Large expert blocks split into finer expert tiles.

Table 8.2: Expert count, hidden dimension, total parameters, and active parameters.

Item	Planned content	Status
Mechanism	Show how more smaller experts can replace fewer larger experts while preserving readable code.	Placeholder
Mini-example	Compare 4 large experts with hidden-dim=1024 to 16 smaller experts with hidden-dim=256.	Placeholder
Diagnostic	Compute total and active expert parameters for several segmentations.	Placeholder

8.3 The MiniDeepSeekMoE layer contract

Define the final layer interface that later chapters assemble into the full model.

Use $D=128$, $E=4$ routed experts, one shared expert, and $K=2$ in the smoke config.

The planned mechanics should introduce the equation before the implementation listing.

$$y = x + \text{shared-out} + \text{routed-out} \text{ when the block is shown with residual context.} \quad (8.3)$$

Listing 8.3: Planned code placeholder for the MiniDeepSeekMoE layer contract.

```

def the_minideepseekmoe_layer_contract_placeholder(x, config):
    """Chapter 8.3 placeholder.

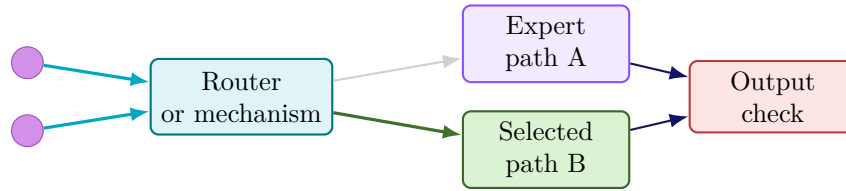
    Planned purpose: MiniDeepSeekMoELayer __init__ showing router, routed experts, and shared experts.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Assert layer output shape equals input shape under smoke and default configs.

The forward pass computes shared and routed paths side by side.



Ch. 8.3: The MiniDeepSeekMoE layer contract

Figure 8.3: MiniDeepSeekMoE layer block with router, routed experts, shared experts, combine, and residual output.

Table 8.3: Final layer tensor contract and configuration fields.

Item	Planned content	Status
Mechanism	Define the final layer interface that later chapters assemble into the full model.	Placeholder
Mini-example	Use D=128, E=4 routed experts, one shared expert, and K=2 in the smoke config.	Placeholder
Diagnostic	Assert layer output shape equals input shape under smoke and default configs.	Placeholder

8.4 Combining shared and routed outputs

Implement the forward pass that sums shared expert output and weighted routed output.

Trace one token through shared expert s0 and routed experts e2/e7.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{moe-out-t} = \text{shared-out-t} + \sum_j \text{gate-}\{tj\} \text{routed-expert-}\{\text{idx-}\{tj\}\}(x-t). \quad (8.4)$$

Listing 8.4: Planned code placeholder for combining shared and routed outputs.

```

def combining_shared_and_routed_outputs_placeholder(x, config):
    """Chapter 8.4 placeholder.

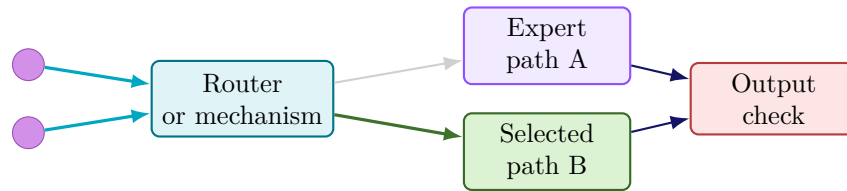
    Planned purpose: Forward pass for shared+routed MoE layer.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Ablate `shared_out` to zero and `routed_out` to zero to confirm both paths affect output.

With two expert paths, diagnostics should separate shared and routed behavior.



Ch. 8.4: Combining shared and routed outputs

Figure 8.4: Shared output and routed weighted output merged before projection back to the decoder block.

Table 8.4: Output components: shared-out, routed-out, residual, and final block output.

Item	Planned content	Status
Mechanism	Implement the forward pass that sums shared expert output and weighted routed output.	Placeholder
Mini-example	Trace one token through shared expert s0 and routed experts e2/e7.	Placeholder
Diagnostic	Ablate shared-out to zero and routed-out to zero to confirm both paths affect output.	Placeholder

8.5 Specialization diagnostics

Plan diagnostics that reveal whether routed experts are used differently across tokens.

Group tokens by simple categories or positions and inspect selected expert frequencies.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Conditional usage frequency } p(\text{expert} \mid \text{token-group}). \quad (8.5)$$

Listing 8.5: Planned code placeholder for specialization diagnostics.

```

def specialization_diagnostics_placeholder(x, config):
    """Chapter 8.5 placeholder.

    Planned purpose: collect_expert_usage_by_token helper.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Generate expert usage heatmaps without overclaiming semantic specialization.

The layer architecture is now complete; the next chapter replaces auxiliary loss with dynamic router bias balancing.

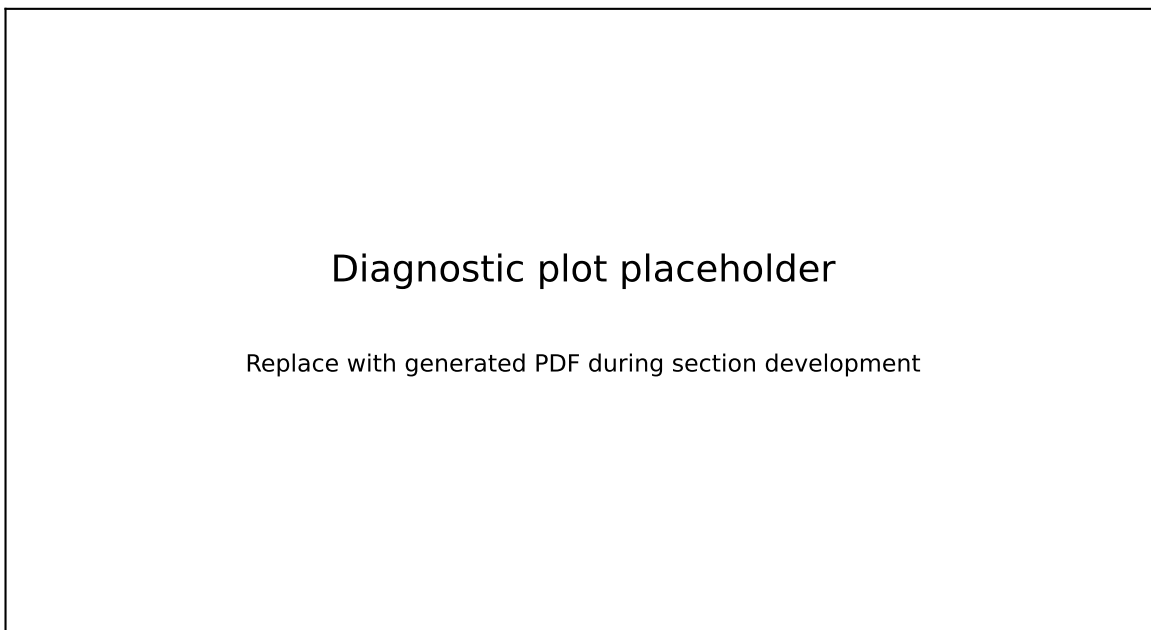


Figure 8.5: Expert usage heatmap by token group or position bucket.

Table 8.5: Specialization probes: token string, position, loss contribution, selected experts.

Item	Planned content	Status
Mechanism	Plan diagnostics that reveal whether routed experts are used differently across tokens.	Placeholder
Mini-example	Group tokens by simple categories or positions and inspect selected expert frequencies.	Placeholder
Diagnostic	Generate expert usage heatmaps without overclaiming semantic specialization.	Placeholder

8.6 Chapter summary and handoff

Confirm that the final expert structure is implemented before changing balancing strategy.

Run one batch through MiniDeepSeekMoELayer and print all returned metrics.

The planned mechanics should introduce the equation before the implementation listing.

Layer output decomposition into shared and routed components. (8.6)

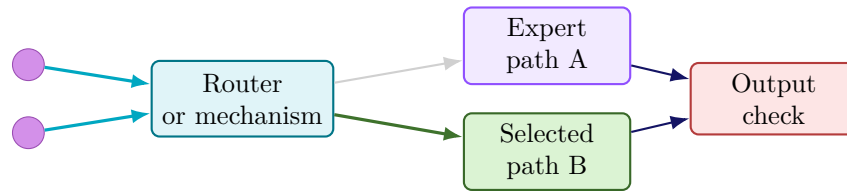
Listing 8.6: Planned code placeholder for chapter summary and handoff.

```
def chapter_summary_and_handoff_placeholder(x, config):
    """Chapter 8.6 placeholder.

    Planned purpose: Smoke test instantiating the final layer contract.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Reader checkpoint: distinguish shared expert compute from routed active compute.



Ch. 8.6: Chapter summary and handoff

Figure 8.6: Completed MiniDeepSeekMoE layer with diagnostic outputs attached.

Table 8.6: Components completed so far and balancing method still used.

Item	Planned content	Status
Mechanism	Confirm that the final expert structure is implemented before changing balancing strategy.	Placeholder
Mini-example	Run one batch through MiniDeepSeekMoELayer and print all returned metrics.	Placeholder
Diagnostic	Reader checkpoint: distinguish shared expert compute from routed active compute.	Placeholder

The next chapter keeps this architecture and changes how expert balance is enforced.

Chapter 9

Auxiliary-loss-free load balancing

This chapter covers

- Why it is useful to separate load-balancing control from the language-model objective.
- How dynamic router bias changes expert selection without becoming a trainable parameter.
- How to compare auxiliary-loss and bias-balancing behavior with identical diagnostics.

Replace auxiliary balancing with a router-bias mechanism that affects selection but not combine weights.

9.1 The limitation of auxiliary balancing

Explain why a separate balancing mechanism is attractive after readers have implemented auxiliary losses.

Show `total_loss` changes when `aux_coef` changes even with the same LM loss.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{total-loss includes auxiliary term in the baseline approach.} \tag{9.1}$$

Listing 9.1: Planned code placeholder for the limitation of auxiliary balancing.

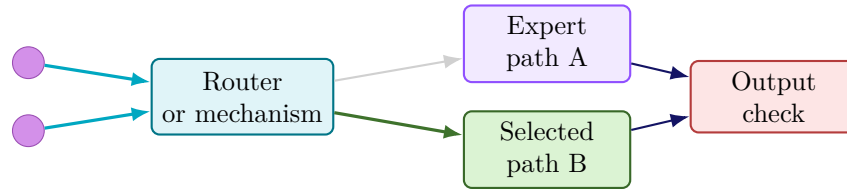
```
def the_limitation_of_auxiliary_balancing_placeholder(x, config):
    """Chapter 9.1 placeholder.

    Planned purpose: Experiment config that toggles aux_loss on and off for the same model.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Compare LM loss and load balance separately, not only total loss.

Router bias moves balance control into the selection rule rather than the loss.



Ch. 9.1: The limitation of auxiliary balancing

Figure 9.1: Auxiliary loss path entering the optimization objective beside LM loss.

Table 9.1: Auxiliary balancing benefits and trade-offs.

Item	Planned content	Status
Mechanism	Explain why a separate balancing mechanism is attractive after readers have implemented auxiliary losses.	Placeholder
Mini-example	Show total-loss changes when aux-coef changes even with the same LM loss.	Placeholder
Diagnostic	Compare LM loss and load balance separately, not only total loss.	Placeholder

9.2 Router bias as non-trainable state

Define router bias as a per-expert control signal updated outside backpropagation.

Use four experts with bias values that rise for underused experts and fall for overused experts.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{selection-score} = \text{score} + \text{router-bias}. \quad (9.2)$$

Listing 9.2: Planned code placeholder for router bias as non trainable state.

```

def router_bias_as_non_trainable_state_placeholder(x, config):
    """Chapter 9.2 placeholder.

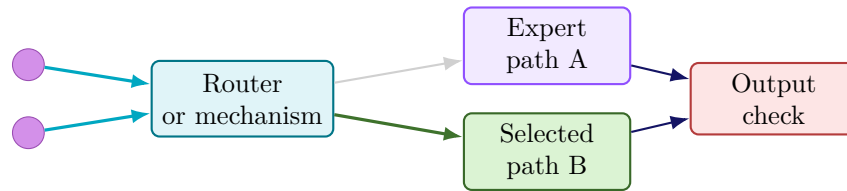
    Planned purpose: Register router_bias as a buffer and exclude it from optimizer gradients.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Assert `router_bias.requires_grad` is false and optimizer does not update it.

The key implementation detail is that bias changes selection but not final gates.



Ch. 9.2: Router bias as non-trainable state

Figure 9.2: Bias vector nudging the score table before top-k selection.

Table 9.2: Router bias properties: shape, initialization, update timing, gradient status, checkpoint behavior.

Item	Planned content	Status
Mechanism	Define router bias as a per-expert control signal updated outside backpropagation.	Placeholder
Mini-example	Use four experts with bias values that rise for underused experts and fall for overused experts.	Placeholder
Diagnostic	Assert router-bias.requires-grad is false and optimizer does not update it.	Placeholder

9.3 Biased selection and unbiased combine

Implement the central invariant of the final reference model.

A token selects experts using score+bias, then gates are computed from the original selected scores.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{idx} = \text{topk}(\text{score} + \text{b}); \text{gate} = \text{normalize}(\text{score}[\text{idx}]). \quad (9.3)$$

Listing 9.3: Planned code placeholder for biased selection and unbiased combine.

```

def biased_selection_and_unbiased_combine_placeholder(x, config):
    """Chapter 9.3 placeholder.

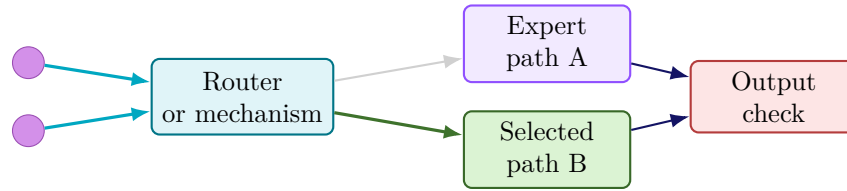
    Planned purpose: Router forward pass returning topk_idx from biased scores and gates from unbiased
    selected scores.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Unit test that changing bias can change expert IDs while gate values still come from raw scores.

The bias vector needs an update rule based on observed expert load.



Ch. 9.3: Biased selection and unbiased combine

Figure 9.3: Two score streams: biased scores for top-k, unbiased scores for gate normalization.

Table 9.3: Selection tensor versus combine tensor and where each is used.

Item	Planned content	Status
Mechanism	Implement the central invariant of the final reference model.	Placeholder
Mini-example	A token selects experts using score+bias, then gates are computed from the original selected scores.	Placeholder
Diagnostic	Unit test that changing bias can change expert IDs while gate values still come from raw scores.	Placeholder

9.4 Dynamic bias update from expert load

Add a simple no-gradient update that nudges underused experts up and overused experts down.

Compare observed loads [8,2,1,1] to target load 3 and update each bias.

The planned mechanics should introduce the equation before the implementation listing.

$$b_i \leftarrow b_i + \gamma \text{ if } \text{load}_i < \text{target}, \text{ otherwise } b_i \leftarrow b_i - \gamma. \quad (9.4)$$

Listing 9.4: Planned code placeholder for dynamic bias update from expert load.

```

def dynamic_bias_update_from_expert_load_placeholder(x, config):
    """Chapter 9.4 placeholder.

    Planned purpose: update_router_bias(load, target, gamma) implemented under torch.no_grad().
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
  
```

Checkpoint

Track bias values over training beside expert loads.

Now we can compare auxiliary balancing and bias balancing using the same model and metrics.

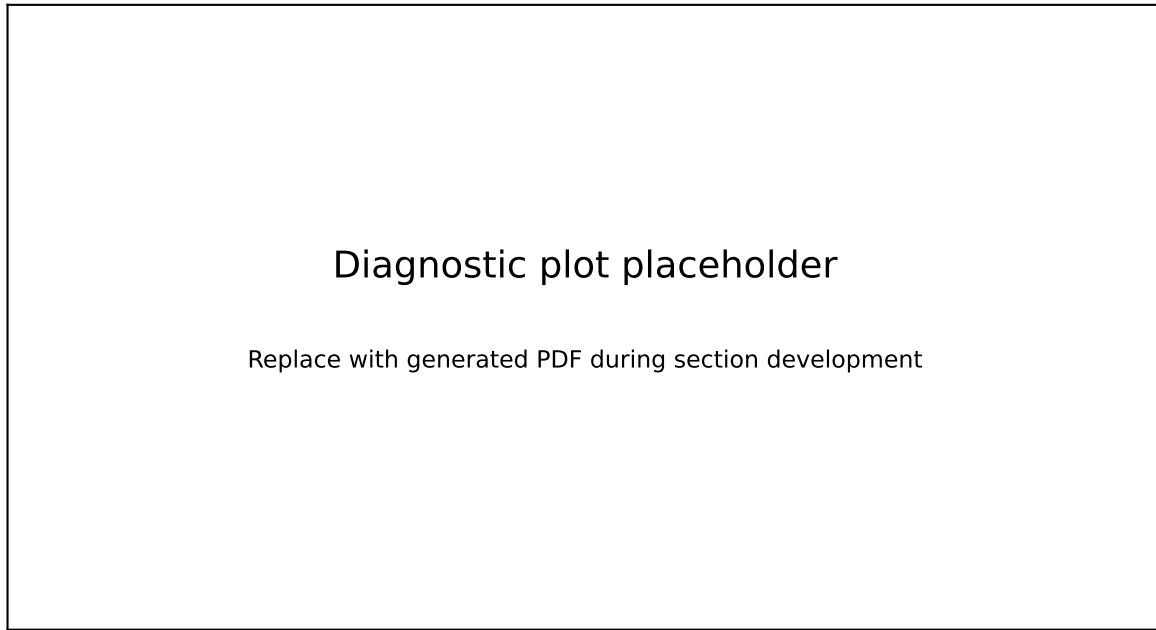


Figure 9.4: Feedback loop from observed expert histogram to router bias update.

Table 9.4: Bias update hyperparameters: gamma, target load, update interval, clamp range.

Item	Planned content	Status
Mechanism	Add a simple no-gradient update that nudges underused experts up and overused experts down.	Placeholder
Mini-example	Compare observed loads [8,2,1,1] to target load 3 and update each bias.	Placeholder
Diagnostic	Track bias values over training beside expert loads.	Placeholder

9.5 Comparing balancing methods

Design a fair chapter experiment comparing no balance, auxiliary loss, and router-bias balancing.

Run three short smoke experiments with identical seeds and report the same diagnostics.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Balance score based on max-load/mean-load or coefficient of variation.} \quad (9.5)$$

Listing 9.5: Planned code placeholder for comparing balancing methods.

```
def comparing_balancing_methods_placeholder(x, config):
    """Chapter 9.5 placeholder.

    Planned purpose: run_balancing_ablation.py command presets.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Produce a table with LM loss, aux loss if used, load balance score, entropy, and drop rate.

The next chapter assembles the full MiniDeepSeekMoE model around this final layer.

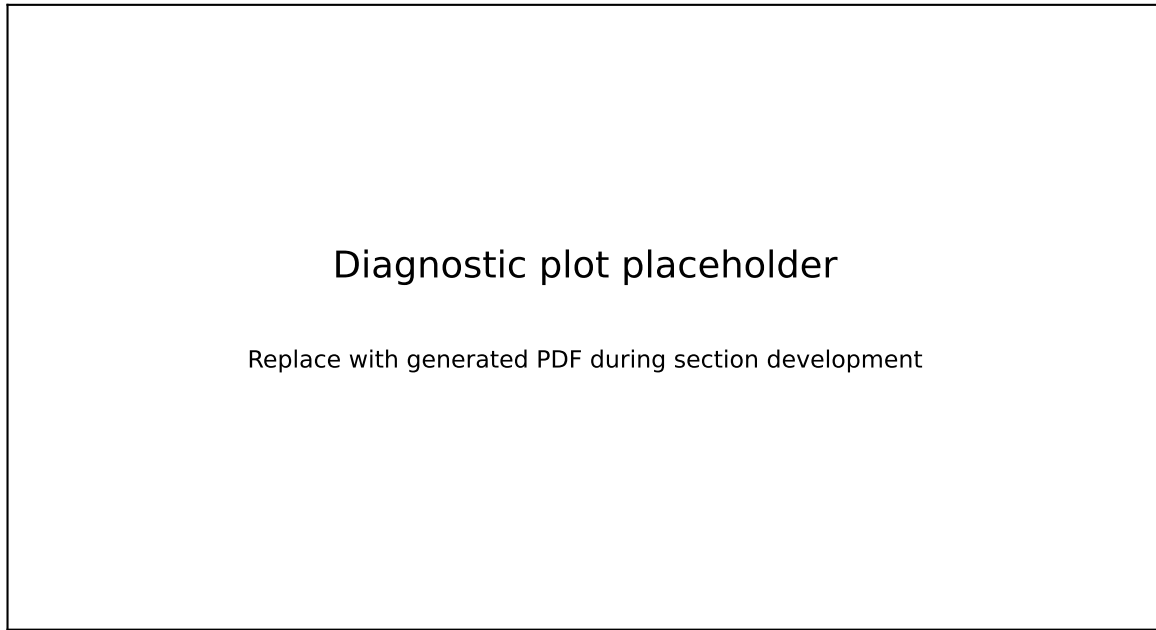


Figure 9.5: Three-panel comparison of expert load histograms and validation loss curves.

Table 9.5: Experiment matrix with balancing method, objective terms, update rule, and expected diagnostics.

Item	Planned content	Status
Mechanism	Design a fair chapter experiment comparing no balance, auxiliary loss, and router-bias balancing.	Placeholder
Mini-example	Run three short smoke experiments with identical seeds and report the same diagnostics.	Placeholder
Diagnostic	Produce a table with LM loss, aux loss if used, load balance score, entropy, and drop rate.	Placeholder

9.6 Chapter summary and handoff

Lock the final router behavior and document the invariants required by the complete model.

Trace one token through score, bias, selected experts, unbiased gates, expert outputs, and metrics.

The planned mechanics should introduce the equation before the implementation listing.

Final routing formula for MiniDeepSeekMoE. (9.6)

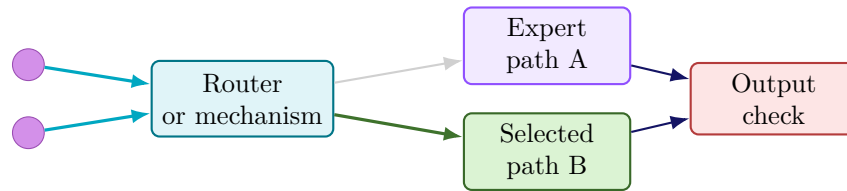
Listing 9.6: Planned code placeholder for chapter summary and handoff.

```
def chapter_summary_and_handoff_placeholder(x, config):
    """Chapter 9.6 placeholder.

    Planned purpose: RouterBiasBalancer smoke test covering update and no-gradient behavior.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Reader checkpoint: explain why bias should not be included in combine weights.



Ch. 9.6: Chapter summary and handoff

Figure 9.6: Final router contract diagram with selection and combine paths separated.

Table 9.6: Final router invariants and tests.

Item	Planned content	Status
Mechanism	Lock the final router behavior and document the invariants required by the complete model.	Placeholder
Mini-example	Trace one token through score, bias, selected experts, unbiased gates, expert outputs, and metrics.	Placeholder
Diagnostic	Reader checkpoint: explain why bias should not be included in combine weights.	Placeholder

We can now build the full model with this routing block as the FFN replacement.

Chapter 10

Assembling MiniDeepSeekMoE end to end

This chapter covers

- How configuration objects control dense, top-k, shared expert, and balancing options.
- How the decoder block returns logits, losses, and routing metrics cleanly.
- How smoke tests protect the full model from hidden routing regressions.

Integrate the final MoE layer into a complete decoder-only language model.

10.1 Configuration objects and model variants

Move from chapter-specific modules to a unified configuration-driven model.

Define `DenseBaselineConfig`, `MiniDeepSeekMoESmokeConfig`, and `MiniDeepSeekMoE16x2Config`.

The planned mechanics should introduce the equation before the implementation listing.

Variant identity as a tuple of depth, width, expert count, K, and balancing method. (10.1)

Listing 10.1: Planned code placeholder for configuration objects and model variants.

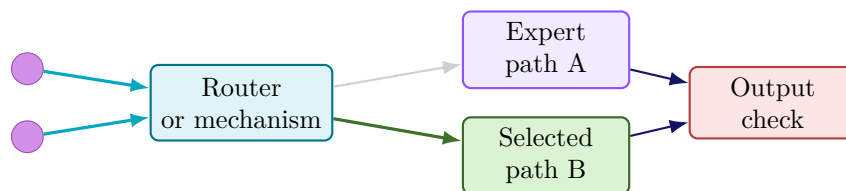
```
def configuration_objects_and_model_variants_placeholder(x, config):
    """Chapter 10.1 placeholder.

    Planned purpose: Dataclass or simple config object for MiniDeepSeekMoE.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Print model summary and assert required fields exist.

The decoder block can now instantiate dense or MoE FFN paths from config.



Ch. 10.1: Configuration objects and model variants

Figure 10.1: Configuration tree controlling attention, FFN/MoE, routing, balancing, and diagnostics.

Table 10.1: Final configuration fields and default values.

Item	Planned content	Status
Mechanism	Move from chapter-specific modules to a unified configuration-driven model.	Placeholder
Mini-example	Define DenseBaselineConfig, MiniDeepSeekMoESmokeConfig, and MiniDeepSeekMoE16x2Config.	Placeholder
Diagnostic	Print model summary and assert required fields exist.	Placeholder

10.2 Decoder block with dense prefix and MoE layers

Assemble layers so early dense computation can precede sparse MoE layers if configured.

Use one dense prefix layer followed by MoE blocks in the default config.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{block}(x) = x + \text{attention}(\text{norm}(x)); x = x + \text{ffn-or-moe}(\text{norm}(x)). \quad (10.2)$$

Listing 10.2: Planned code placeholder for decoder block with dense prefix and MoE layers.

```

def decoder_block_with_dense_prefix_and_moe_layers_placeholder(x, config):
    """Chapter 10.2 placeholder.

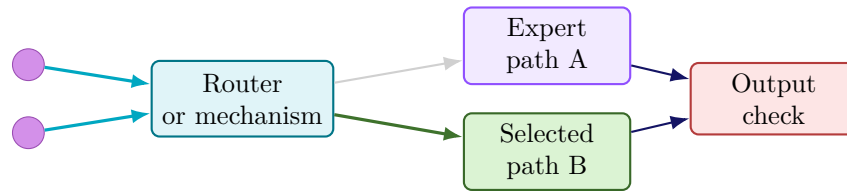
    Planned purpose: DecoderBlock factory that chooses DenseFFN or MiniDeepSeekMoELayer.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

List layer types during model initialization.

The model forward pass must carry routing metrics out of each MoE layer.



Ch. 10.2: Decoder block with dense prefix and MoE layers

Figure 10.2: Stack of decoder blocks showing dense prefix and repeated MoE layers.

Table 10.2: Layer type by index for smoke, default, and larger teaching configs.

Item	Planned content	Status
Mechanism	Assemble layers so early dense computation can precede sparse MoE layers if configured.	Placeholder
Mini-example	Use one dense prefix layer followed by MoE blocks in the default config.	Placeholder
Diagnostic	List layer types during model initialization.	Placeholder

10.3 Forward pass outputs and loss dictionary

Return logits and structured losses/metrics without making training code guess where values live.

Forward pass returns logits, `lm_loss`, `total_loss`, `router_metrics`, and optional `aux_loss`.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{total-loss depends on selected balancing mode.} \quad (10.3)$$

Listing 10.3: Planned code placeholder for forward pass outputs and loss dictionary.

```

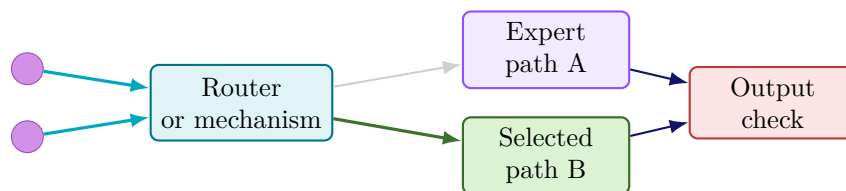
def forward_pass_outputs_and_loss_dictionary_placeholder(x, config):
    """Chapter 10.3 placeholder.

    Planned purpose: MiniDeepSeekMoE.forward implementation with metric aggregation.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
  
```

Checkpoint

Unit test for output keys under dense, auxiliary, and bias modes.

Training and sampling scripts can now be shared across variants.



Ch. 10.3: Forward pass outputs and loss dictionary

Figure 10.3: Forward output dictionary with model outputs and nested per-layer router metrics.

Table 10.3: Output dictionary keys, shapes, and when they are present.

Item	Planned content	Status
Mechanism	Return logits and structured losses/metrics without making training code guess where values live.	Placeholder
Mini-example	Forward pass returns logits, lm-loss, total-loss, router-metrics, and optional aux-loss.	Placeholder
Diagnostic	Unit test for output keys under dense, auxiliary, and bias modes.	Placeholder

10.4 Training and sampling scripts

Create runnable scripts that match the source codebase style: minimal, readable, and chapter-scoped.

Run `train.py` with a smoke config and `sample.py` from the saved checkpoint.

The planned mechanics should introduce the equation before the implementation listing.

Checkpoint state includes model, optimizer, config, and router bias buffers. (10.4)

Listing 10.4: Planned code placeholder for training and sampling scripts.

```

def training_and_sampling_scripts_placeholder(x, config):
    """Chapter 10.4 placeholder.

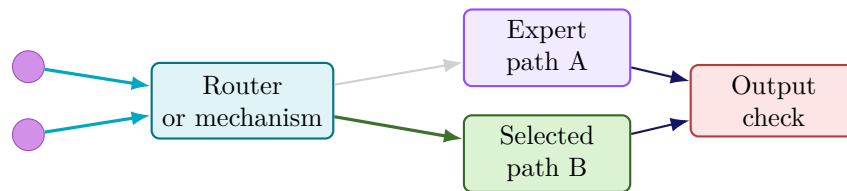
    Planned purpose: train.py main function with config loading, training loop, checkpointing, and
    sampling hook.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

Verify checkpoint reload produces logits of the same shape.

A full model needs full-model smoke tests before experiments are trusted.



Ch. 10.4: Training and sampling scripts

Figure 10.4: Script workflow from config to training to checkpoint to sample text.

Table 10.4: Command-line arguments and their defaults.

Item	Planned content	Status
Mechanism	Create runnable scripts that match the source codebase style: minimal, readable, and chapter-scoped.	Placeholder
Mini-example	Run train.py with a smoke config and sample.py from the saved checkpoint.	Placeholder
Diagnostic	Verify checkpoint reload produces logits of the same shape.	Placeholder

10.5 Full-model smoke tests

Create a test suite that catches shape, routing, balancing, and serialization failures.

Run the smoke config on CPU for two forward/backward passes.

The planned mechanics should introduce the equation before the implementation listing.

No NaNs and finite loss invariant. (10.5)

Listing 10.5: Planned code placeholder for full model smoke tests.

```

def full_model_smoke_tests_placeholder(x, config):
    """Chapter 10.5 placeholder.

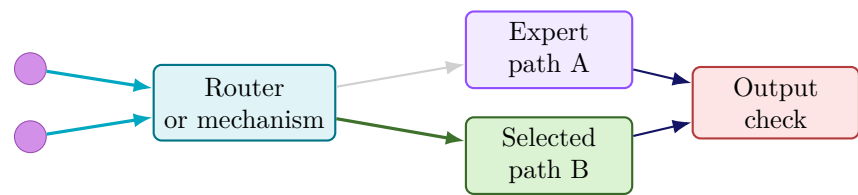
    Planned purpose: tests/test_minideepseekmoe_smoke.py with forward, backward, bias-update, and save-
    load tests.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")

```

Checkpoint

CI-style console summary for all smoke tests.

With a complete tested implementation, the next chapter runs experiments and interprets diagnostics.



Ch. 10.5: Full-model smoke tests

Figure 10.5: Smoke-test pipeline with pass/fail checkpoints after each stage.

Table 10.5: Test names, purpose, and expected runtime tier.

Item	Planned content	Status
Mechanism	Create a test suite that catches shape, routing, balancing, and serialization failures.	Placeholder
Mini-example	Run the smoke config on CPU for two forward/backward passes.	Placeholder
Diagnostic	CI-style console summary for all smoke tests.	Placeholder

10.6 Chapter summary and handoff

Summarize the complete implementation and separate code correctness from model quality.
Compare dense baseline and MiniDeepSeekMoE smoke configs by parameter count and active parameters.
The planned mechanics should introduce the equation before the implementation listing.

Active expert parameters per token for the final config.

(10.6)

Listing 10.6: Planned code placeholder for chapter summary and handoff.

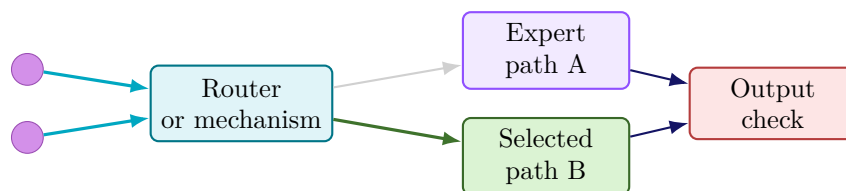
```
def chapter_summary_and_handoff_placeholder(x, config):
    """Chapter 10.6 placeholder.

    Planned purpose: README quickstart commands for train and sample.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Reader checkpoint: run the smoke model and inspect router metrics.

The next chapter turns the implementation into a controlled experiment framework.



Ch. 10.6: Chapter summary and handoff

Figure 10.6: End-to-end model architecture diagram from tokens to logits.

Table 10.6: Final implementation files and their responsibilities.

Item	Planned content	Status
Mechanism	Summarize the complete implementation and separate code correctness from model quality.	Placeholder
Mini-example	Compare dense baseline and MiniDeepSeekMoE smoke configs by parameter count and active parameters.	Placeholder
Diagnostic	Reader checkpoint: run the smoke model and inspect router metrics.	Placeholder

Chapter 11

Training, diagnostics, and controlled experiments

This chapter covers

- How to run small, reproducible experiments without overclaiming performance.
- How to interpret routing diagnostics alongside language-model loss.
- How ablations reveal the contribution of top-k, shared experts, and balancing methods.

Use the model to answer concrete questions about routing, balance, and sparse capacity.

11.1 Experiment design for a from-scratch book

Set realistic expectations for small-data experiments and define what can and cannot be concluded.

Use three smoke-scale runs to compare mechanics, not state-of-the-art quality.

The planned mechanics should introduce the equation before the implementation listing.

Report mean and range across seeds when multiple seeds are used. (11.1)

Listing 11.1: Planned code placeholder for experiment design for a from scratch book.

```
def experiment_design_for_a_from_scratch_book_placeholder(x, config):  
    """Chapter 11.1 placeholder.  
  
    Planned purpose: Experiment manifest YAML with seeds, configs, and output directories.  
    Replace this stub during section development.  
    """  
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Check that every experiment saves config, seed, logs, and git/code snapshot note.

The data pipeline needs a repeatable preparation script.

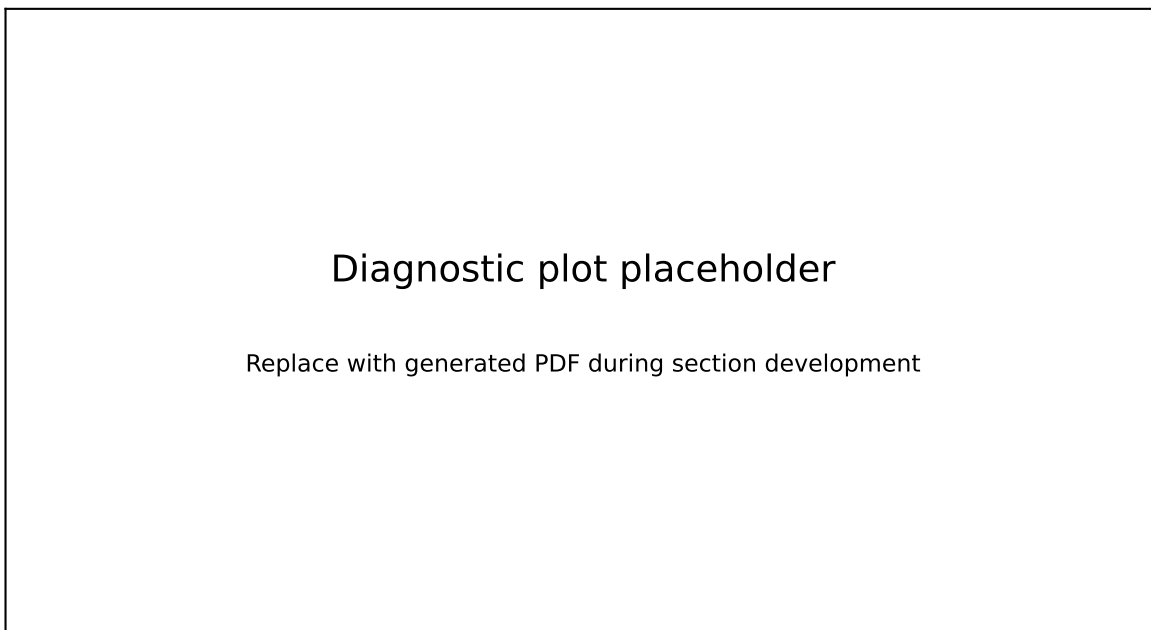


Figure 11.1: Experiment funnel from smoke tests to short training runs to optional longer ablations.

Table 11.1: Claim types allowed by each experiment scale.

Item	Planned content	Status
Mechanism	Set realistic expectations for small-data experiments and define what can and cannot be concluded.	Placeholder
Mini-example	Use three smoke-scale runs to compare mechanics, not state-of-the-art quality.	Placeholder
Diagnostic	Check that every experiment saves config, seed, logs, and git/code snapshot note.	Placeholder

11.2 Dataset preparation and reproducibility

Make dataset setup deterministic and lightweight enough for readers to rerun.

Prepare a small corpus split into train and validation binary or tensor files.

The planned mechanics should introduce the equation before the implementation listing.

Train/validation split ratio and token count definitions. (11.2)

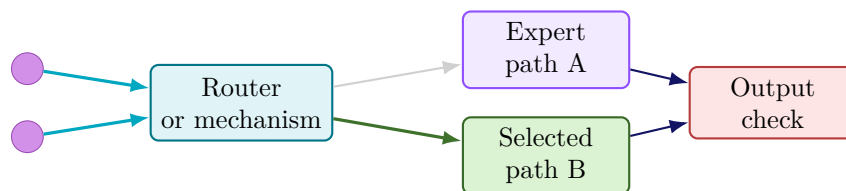
Listing 11.2: Planned code placeholder for dataset preparation and reproducibility.

```
def dataset_preparation_and_reproducibility_placeholder(x, config):
    """Chapter 11.2 placeholder.

    Planned purpose: prepare.py script with deterministic split and saved metadata.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Print token counts, split sizes, and a decoded sample from each split.



Ch. 11.2: Dataset preparation and reproducibility

Figure 11.2: Raw text to tokenized train/validation artifacts.

Table 11.2: Dataset artifacts, paths, and regeneration commands.

Item	Planned content	Status
Mechanism	Make dataset setup deterministic and lightweight enough for readers to rerun.	Placeholder
Mini-example	Prepare a small corpus split into train and validation binary or tensor files.	Placeholder
Diagnostic	Print token counts, split sizes, and a decoded sample from each split.	Placeholder

Training logs must include both language and routing metrics.

11.3 Loss curves and routing dashboards

Create consistent plots that make model behavior inspectable after every run.

Plot validation loss, load balance score, entropy, and drop rate from one JSONL log.

The planned mechanics should introduce the equation before the implementation listing.

Load-balance score based on coefficient of variation across expert loads. (11.3)

Listing 11.3: Planned code placeholder for loss curves and routing dashboards.

```
def loss_curves_and_routing_dashboards_placeholder(x, config):  
    """Chapter 11.3 placeholder.  
  
    Planned purpose: plot_experiment_dashboard.py reading logs and saving PDF figures.  
    Replace this stub during section development.  
    """  
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Save dashboard figures through LaTeX wrappers, never raw includegraphics in prose.

With dashboards ready, ablation experiments can compare design choices.

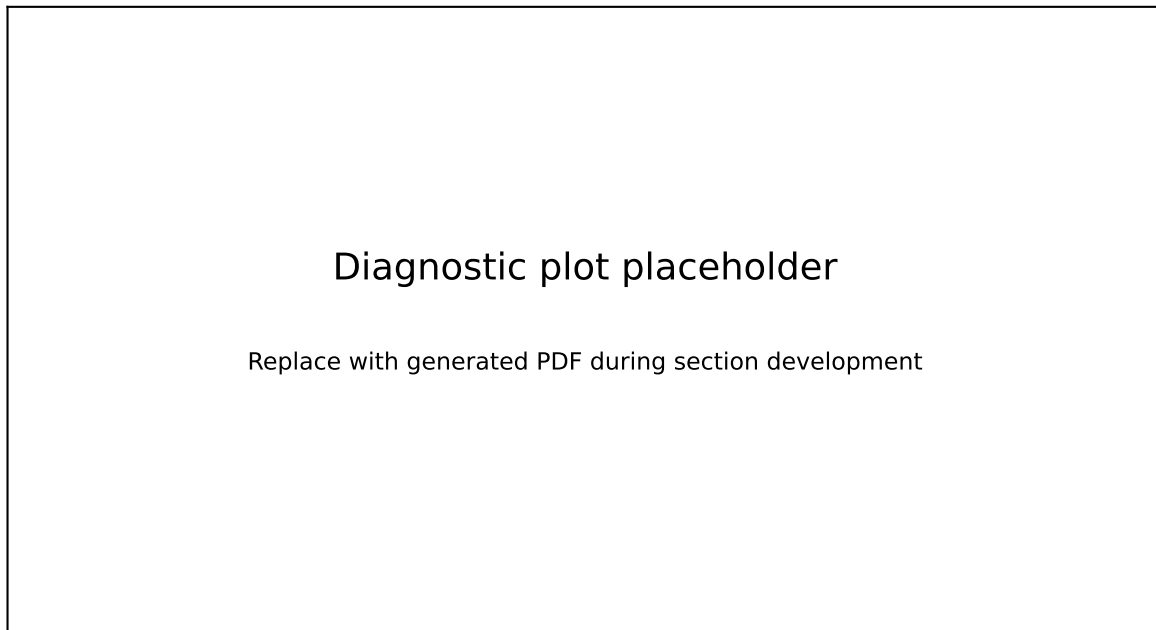


Figure 11.3: Standard diagnostic dashboard layout for a chapter experiment.

Table 11.3: Plot names, source metrics, and interpretation cautions.

Item	Planned content	Status
Mechanism	Create consistent plots that make model behavior inspectable after every run.	Placeholder
Mini-example	Plot validation loss, load balance score, entropy, and drop rate from one JSONL log.	Placeholder
Diagnostic	Save dashboard figures through LaTeX wrappers, never raw includegraphics in prose.	Placeholder

11.4 Ablation matrix: dense, top-1, top-2, shared, and bias-balanced

Plan the central comparison table for the book without pretending small runs settle model quality.

Run five variants for the same token budget and record comparable metrics.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Active parameter ratio for each sparse variant.} \quad (11.4)$$

Listing 11.4: Planned code placeholder for ablation matrix dense top 1 top 2 shared and bias balanced.

```
def ablation_matrix_dense_top_1_top_2_shared_and_bias_balanced_placeholder(x, config):
    """Chapter 11.4 placeholder.

    Planned purpose: run_ablation_matrix.py that launches or documents variant commands.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Flag any run with unstable loss, high drop rate, or extreme expert collapse.

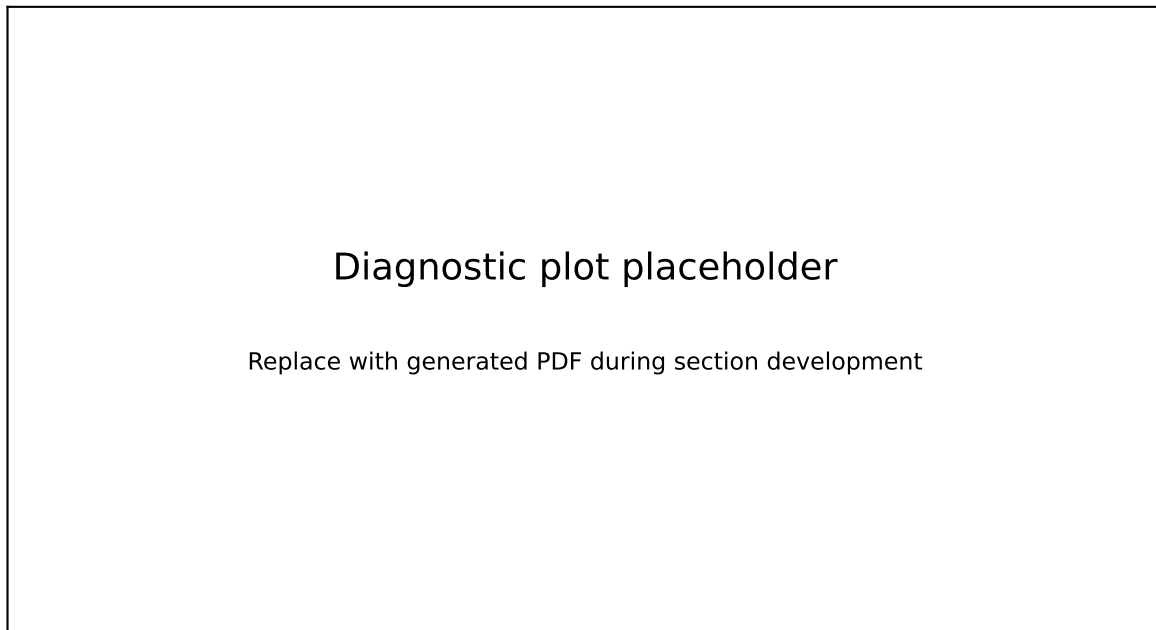


Figure 11.4: Ablation matrix with one row per model variant and columns for routing features.

Table 11.4: Dense baseline versus MoE variants: parameters, active parameters, loss, load, entropy, drop rate.

Item	Planned content	Status
Mechanism	Plan the central comparison table for the book without pretending small runs settle model quality.	Placeholder
Mini-example	Run five variants for the same token budget and record comparable metrics.	Placeholder
Diagnostic	Flag any run with unstable loss, high drop rate, or extreme expert collapse.	Placeholder

Expert behavior should be interpreted with both numbers and examples.

11.5 Interpreting expert behavior carefully

Teach readers to inspect expert usage without overclaiming that tiny experts learned semantic roles.

Look at expert IDs selected for a short prompt and compare across positions or token types.

The planned mechanics should introduce the equation before the implementation listing.

Conditional expert frequency for a chosen token group. (11.5)

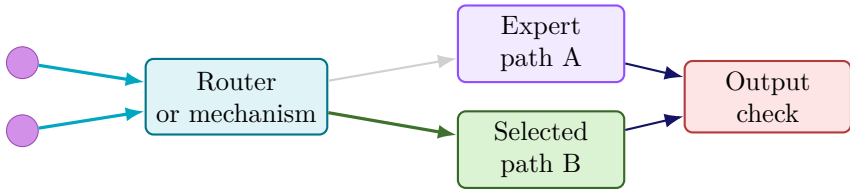
Listing 11.5: Planned code placeholder for interpreting expert behavior carefully.

```
def interpreting_expert_behavior_carefully_placeholder(x, config):  
    """Chapter 11.5 placeholder.  
  
    Planned purpose: trace_prompt_routing.py that prints token, top-k experts, and gates.  
    Replace this stub during section development.  
    """  
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Generate a routing trace table and a usage heatmap.

The final chapter steps back from experiments to discuss inference and scaling limits.



Ch. 11.5: Interpreting expert behavior carefully

Figure 11.5: Token-by-token routing trace over a prompt.

Table 11.5: Interpretation checklist: evidence, limitation, alternative explanation, next test.

Item	Planned content	Status
Mechanism	Teach readers to inspect expert usage without over-claiming that tiny experts learned semantic roles.	Placeholder
Mini-example	Look at expert IDs selected for a short prompt and compare across positions or token types.	Placeholder
Diagnostic	Generate a routing trace table and a usage heatmap.	Placeholder

11.6 Chapter summary and handoff

Close the experimental part by identifying what the book has demonstrated mechanically.
Summarize one dense run and one final MoE run in the same report card.
The planned mechanics should introduce the equation before the implementation listing.

No new equation; reuse active-parameter and load-balance metrics. (11.6)

Listing 11.6: Planned code placeholder for chapter summary and handoff.

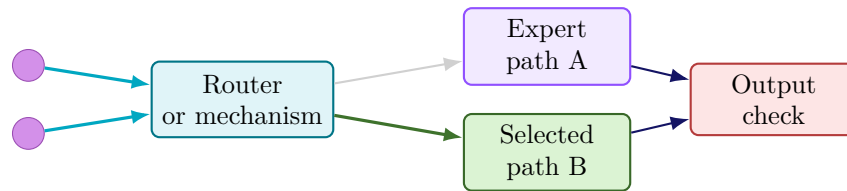
```
def chapter_summary_and_handoff_placeholder(x, config):
    """Chapter 11.6 placeholder.

    Planned purpose: generate_experiment_report.py producing Markdown and LaTeX table outputs.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Reader checkpoint: identify whether a routing issue is code, loss, capacity, or data related.

The last chapter explains how these same mechanisms change at inference and production scale.



Ch. 11.6: Chapter summary and handoff

Figure 11.6: Experiment report card combining loss, active parameters, expert balance, and routing trace.

Table 11.6: What each ablation teaches and which chapter implemented the mechanism.

Item	Planned content	Status
Mechanism	Close the experimental part by identifying what the book has demonstrated mechanically.	Placeholder
Mini-example	Summarize one dense run and one final MoE run in the same report card.	Placeholder
Diagnostic	Reader checkpoint: identify whether a routing issue is code, loss, capacity, or data related.	Placeholder

Chapter 12

Inference, performance, and scaling beyond the toy model

This chapter covers

- How MoE affects inference flow, batching, and active-parameter accounting.
- Why expert parallelism and all-to-all communication appear at larger scale.
- How to map MiniDeepSeekMoE concepts to production-scale MoE models responsibly.

Connect the from-scratch implementation to systems concerns without turning the book into a distributed-systems text.

12.1 Autoregressive inference with sparse FFNs

Explain what changes and what stays the same when a trained MoE model generates text one token at a time.

Generate one new token and trace attention, router, selected experts, and logits.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Per-step active expert calls equal layers-with-moe} * K \text{ for one token.} \quad (12.1)$$

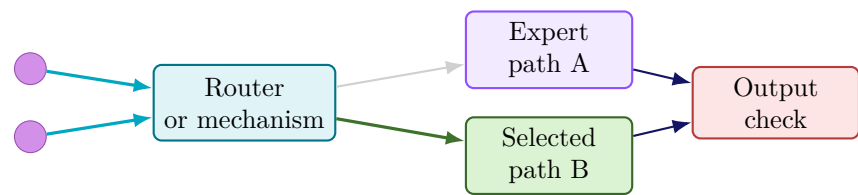
Listing 12.1: Planned code placeholder for autoregressive inference with sparse FFNs.

```
def autoregressive_inference_with_sparse_ffns_placeholder(x, config):  
    """Chapter 12.1 placeholder.  
  
    Planned purpose: generate function with routing metrics optionally returned per step.  
    Replace this stub during section development.  
    """  
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Print selected experts for each generated token in a short sample.

Batching multiple requests complicates the expert batches.



Ch. 12.1: Autoregressive inference with sparse FFNs

Figure 12.1: One-token inference path through attention cache and MoE layer.

Table 12.1: Training-time versus inference-time routing tensors.

Item	Planned content	Status
Mechanism	Explain what changes and what stays the same when a trained MoE model generates text one token at a time.	Placeholder
Mini-example	Generate one new token and trace attention, router, selected experts, and logits.	Placeholder
Diagnostic	Print selected experts for each generated token in a short sample.	Placeholder

12.2 Batching routed tokens at inference

Show why MoE inference wants tokens for the same expert to be grouped efficiently.
Two prompts in a batch route their next token to different experts.
The planned mechanics should introduce the equation before the implementation listing.

Expert batch size distribution at generation step s .

(12.2)

Listing 12.2: Planned code placeholder for batching routed tokens at inference.

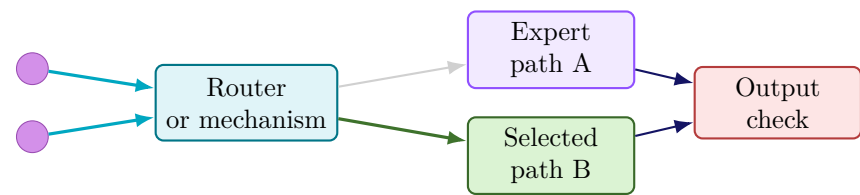
```
def batching_routed_tokens_at_inference_placeholder(x, config):
    """Chapter 12.2 placeholder.

    Planned purpose: profile_inference_routes function collecting per-step expert batch sizes.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Histogram expert batch sizes over a generated sequence.

Compute and memory estimates clarify where the toy implementation stops scaling.



Ch. 12.2: Batching routed tokens at inference

Figure 12.2: Inference batch tokens packed into expert-specific microbatches.

Table 12.2: Inference batching challenges: uneven loads, small expert batches, padding, latency.

Item	Planned content	Status
Mechanism	Show why MoE inference wants tokens for the same expert to be grouped efficiently.	Placeholder
Mini-example	Two prompts in a batch route their next token to different experts.	Placeholder
Diagnostic	Histogram expert batch sizes over a generated sequence.	Placeholder

12.3 Compute, memory, and active-parameter accounting

Give readers a sober way to discuss total parameters, active parameters, and practical memory costs.
Compare dense FFN, 16x2 MoE, and 64x6 teaching config parameter counts.
The planned mechanics should introduce the equation before the implementation listing.

Active expert FFN parameters per token approximately K times one routed expert plus shared experts. (12.3)

Listing 12.3: Planned code placeholder for compute memory and active parameter accounting.

```
def compute_memory_and_active_parameter_accounting_placeholder(x, config):
    """Chapter 12.3 placeholder.

    Planned purpose: estimate_moe_compute.py producing parameter and active-compute estimates.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Check estimates against `model.count_parameters` breakdown.

At production scale, experts are often spread across devices, introducing communication.

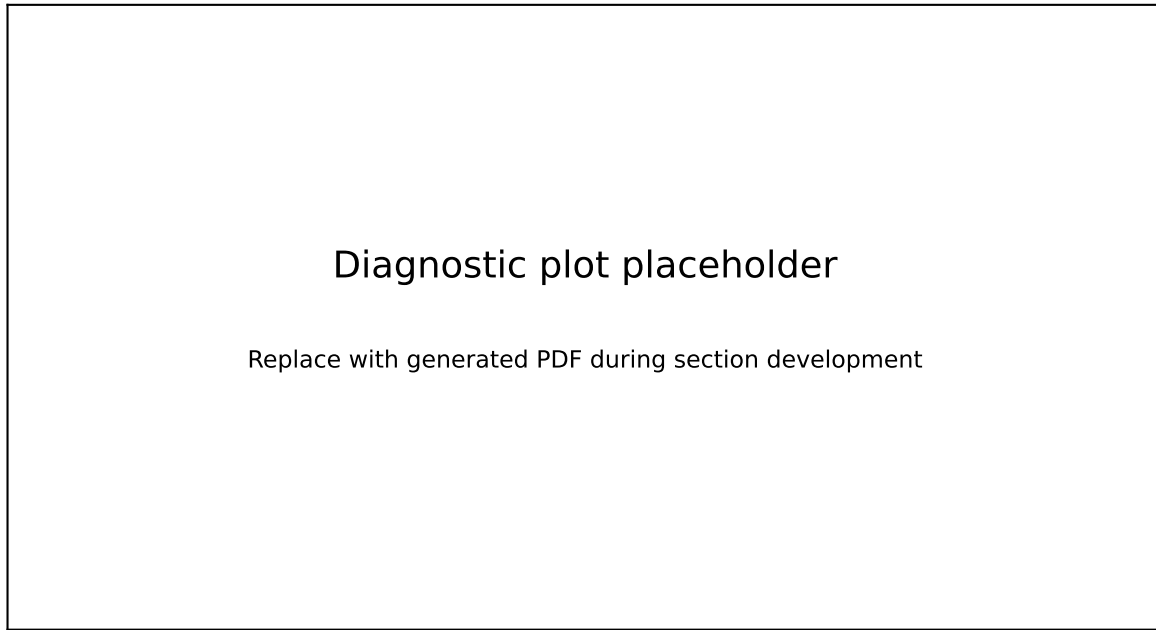


Figure 12.3: Stacked bars for total parameters and active parameters per token.

Table 12.3: Total parameters, active parameters, estimated activation memory, and expert calls by config.

Item	Planned content	Status
Mechanism	Give readers a sober way to discuss total parameters, active parameters, and practical memory costs.	Placeholder
Mini-example	Compare dense FFN, 16x2 MoE, and 64x6 teaching config parameter counts.	Placeholder
Diagnostic	Check estimates against model.count-parameters breakdown.	Placeholder

12.4 Expert parallelism and all-to-all communication

Explain the systems picture conceptually while keeping implementation out of scope.

Two devices each host two experts; tokens must move to the device containing their selected experts.

The planned mechanics should introduce the equation before the implementation listing.

$$\text{Communication volume depends on routed token count, } D, K, \text{ and bytes per element.} \quad (12.4)$$

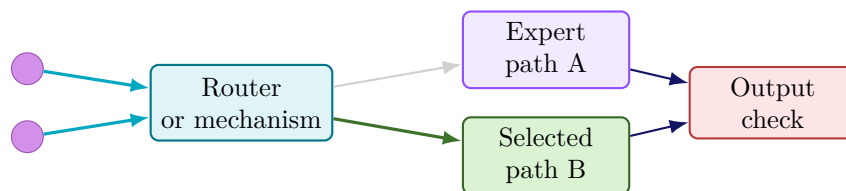
Listing 12.4: Planned code placeholder for expert parallelism and all to all communication.

```
def expert_parallelism_and_all_to_all_communication_placeholder(x, config):
    """Chapter 12.4 placeholder.

    Planned purpose: Conceptual pseudocode for distributed dispatch without runnable distributed code.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Thought experiment table estimating routed activation traffic for a small config.



Ch. 12.4: Expert parallelism and all-to-all communication

Figure 12.4: Tokens routed across devices through an all-to-all exchange, expert compute, and return exchange.

Table 12.4: Single-device MoE versus expert-parallel MoE responsibilities.

Item	Planned content	Status
Mechanism	Explain the systems picture conceptually while keeping implementation out of scope.	Placeholder
Mini-example	Two devices each host two experts; tokens must move to the device containing their selected experts.	Placeholder
Diagnostic	Thought experiment table estimating routed activation traffic for a small config.	Placeholder

This systems view helps map the toy model to public large-MoE architecture descriptions.

12.5 Mapping the book model to production MoE families

Connect MiniDeepSeekMoE pieces to broader MoE designs while staying honest about omitted details.

Map top-k routing, shared experts, fine-grained experts, and router bias to the final reference model vocabulary.

The planned mechanics should introduce the equation before the implementation listing.

Scaling identity: same layer contract, larger E, larger D, more devices. (12.5)

Listing 12.5: Planned code placeholder for mapping the book model to production MoE families.

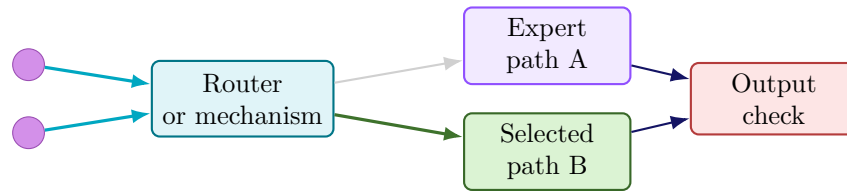
```
def mapping_the_book_model_to_production_moe_families_placeholder(x, config):
    """Chapter 12.5 placeholder.

    Planned purpose: Config comparison snippet showing smoke, default, and larger teaching configs.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
```

Checkpoint

Reader checkpoint: identify which chapter implemented each production analogue.

The book closes by summarizing the reusable mental model.



Ch. 12.5: Mapping the book model to production MoE families

Figure 12.5: Concept map from book components to larger MoE system components.

Table 12.5: Book implementation feature, production analogue, and what remains out of scope.

Item	Planned content	Status
Mechanism	Connect MiniDeepSeekMoE pieces to broader MoE designs while staying honest about omitted details.	Placeholder
Mini-example	Map top-k routing, shared experts, fine-grained experts, and router bias to the final reference model vocabulary.	Placeholder
Diagnostic	Reader checkpoint: identify which chapter implemented each production analogue.	Placeholder

12.6 Book summary and next steps

End with a coherent picture of MoE as a route-execute-combine layer embedded in a decoder.

Trace one token through the final model using the vocabulary from every part of the book.

The planned mechanics should introduce the equation before the implementation listing.

Final MiniDeepSeekMoE layer equation repeated as the book endpoint. (12.6)

Listing 12.6: Planned code placeholder for book summary and next steps.

```

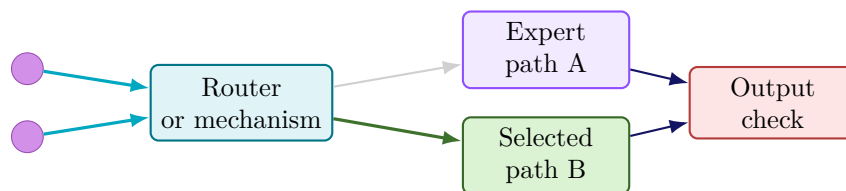
def book_summary_and_next_steps_placeholder(x, config):
    """Chapter 12.6 placeholder.

    Planned purpose: Extension checklist: add RMSNorm, alternate attention, larger datasets, or
    distributed dispatch.
    Replace this stub during section development.
    """
    raise NotImplementedError("Implement this section's code path.")
  
```

Checkpoint

Final self-test: build, train smoke, inspect routing, sample text, and explain limitations.

No next chapter; point readers to appendices and experiments they can extend.



Ch. 12.6: Book summary and next steps

Figure 12.6: Complete route-execute-combine summary diagram with training and inference diagnostics around it.

Table 12.6: Reader capabilities after finishing the book and possible extensions.

Item	Planned content	Status
Mechanism	End with a coherent picture of MoE as a route-execute-combine layer embedded in a decoder.	Placeholder
Mini-example	Trace one token through the final model using the vocabulary from every part of the book.	Placeholder
Diagnostic	Final self-test: build, train smoke, inspect routing, sample text, and explain limitations.	Placeholder

Appendix A

LaTeX artifact and diagram rules

Document the color map, input-only artifact rule, label conventions, and TikZ patterns.

Note

This appendix is planned in Phase 4 and should be expanded during section development.

Appendix B

Tensor shape checklist

Collect every tensor symbol and shape used in routing, dispatch, balancing, and diagnostics.

Note

This appendix is planned in Phase 4 and should be expanded during section development.

Appendix C

Codebase smoke tests and reproducibility checklist

Provide a compact checklist for validating notebooks, scripts, tests, figures, and LaTeX builds.

Note

This appendix is planned in Phase 4 and should be expanded during section development.